School of Computer Science Engineering & Information Systems

Department of Software & Systems Engineering

ISWE406P – Agile Development Process and Devops Lab

ASSESSMENT -3

NAME : SARAVANAN.R

REGISTER NO : 23MIS0460

FACULTY NAME : DR . RAMALINGAM M

EXPERIMENT – 1 :

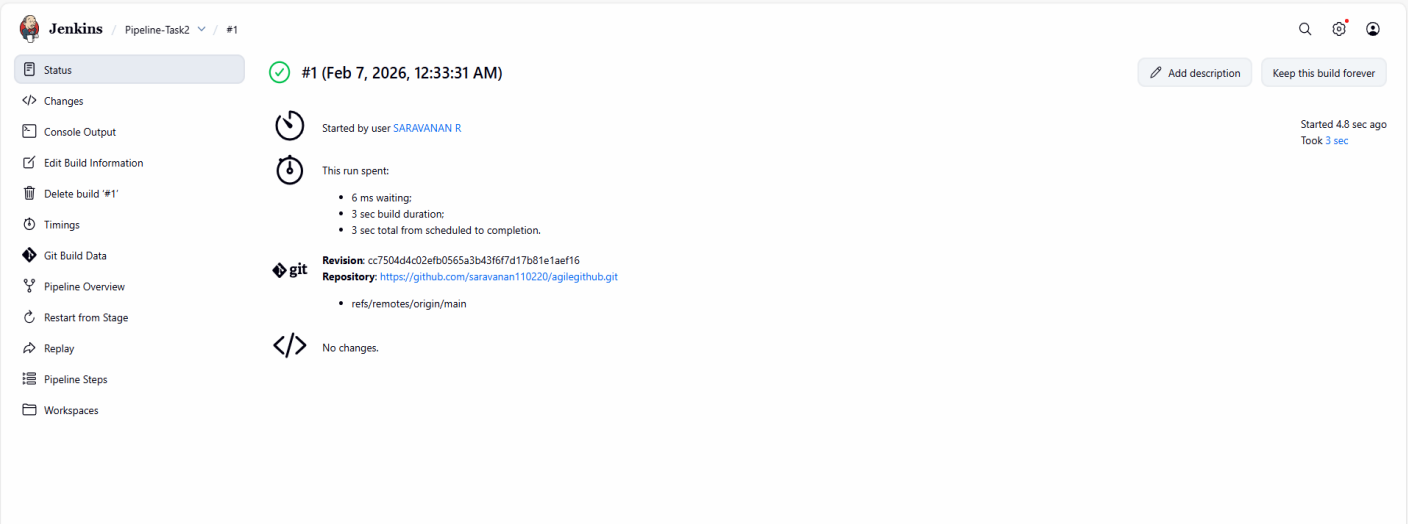
Jenkins CI Pipeline using Git and Maven

Objective:

To implement a Continuous Integration (CI) pipeline using Jenkins that automatically checks out a Maven project from GitHub, builds the project, runs JUnit test cases, and reports the build status.

Tools Used:

- Jenkins
- Git
- GitHub
- Apache Maven



- Java JDK
- Web Browser

<u>Step 1:</u> Create a Simple Maven Project (Git Side)

Project Name:

simple-maven-app

<u>Standard Maven Folder Structure (</u>Mandatory):

```
simple-maven-app/
 ├── pom.xml
 └── src/
    ├── main/java/com/example/App.java
    └── test/java/com/example/AppTest.java
```

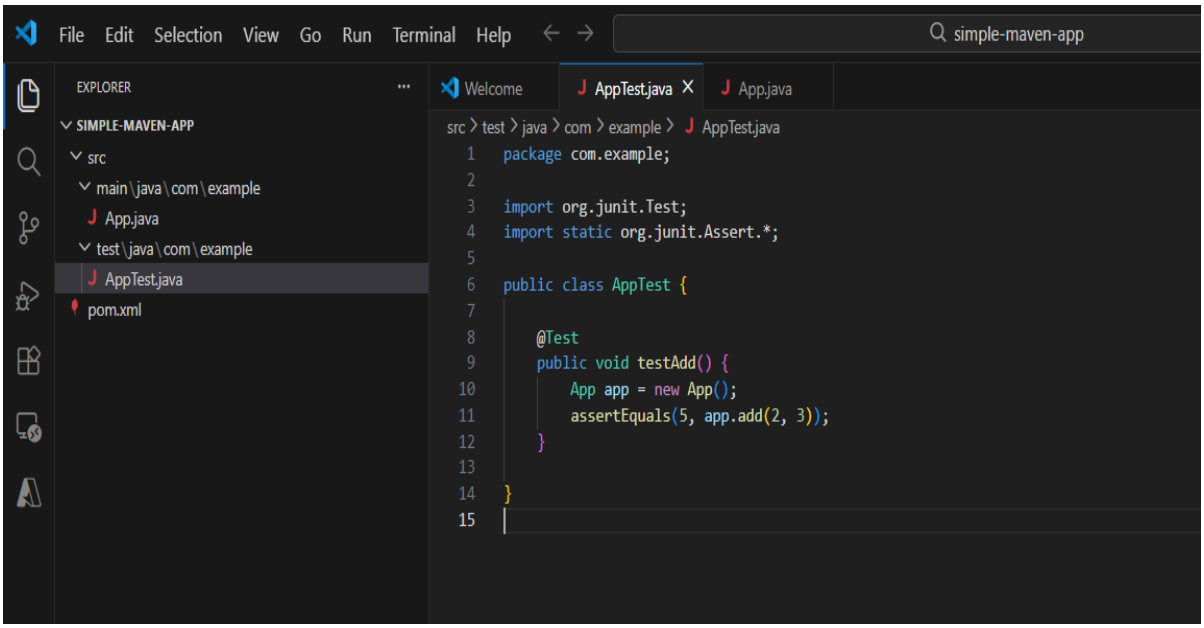


Figure 1: Maven Standard Project Structure

<u>Step 2:</u> Create App.java (Main Application)

File Path:

src/main/java/com/example/App.java

<u>Code:</u>

```
package com.example;
```

```
public class App {
    public int add(int a, int b) {
        return a + b;
    }
}
```

Explanation:

- Defines business logic.
- add() method returns sum of two numbers.
- This method will be tested using JUnit.

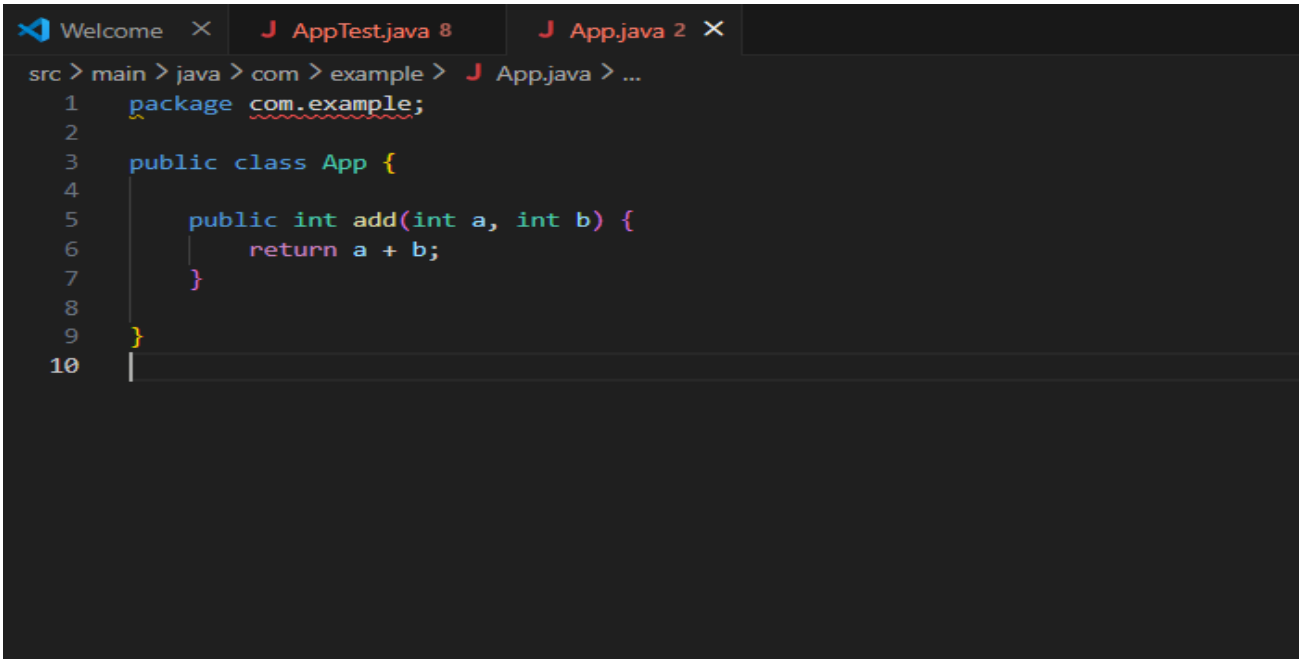


Figure 2: Main Application File (App.java)

<u>Step 3:</u> Create AppTest.java (JUnit Test Case)

File Path:

src/test/java/com/example/AppTest.java

<u>Code:</u>

```
package com.example;


import org.junit.Test;
import static org.junit.Assert.*;


public class AppTest {


    @Test
```

```
    public void testAdd() {
        App app = new App();
        assertEquals(5, app.add(2, 3));
    }
}
```

Explanation:

- @Test marks the method as a test case.
- assertEquals() compares expected and actual values.
- If test fails → Maven build fails.
- If test passes → Build success.

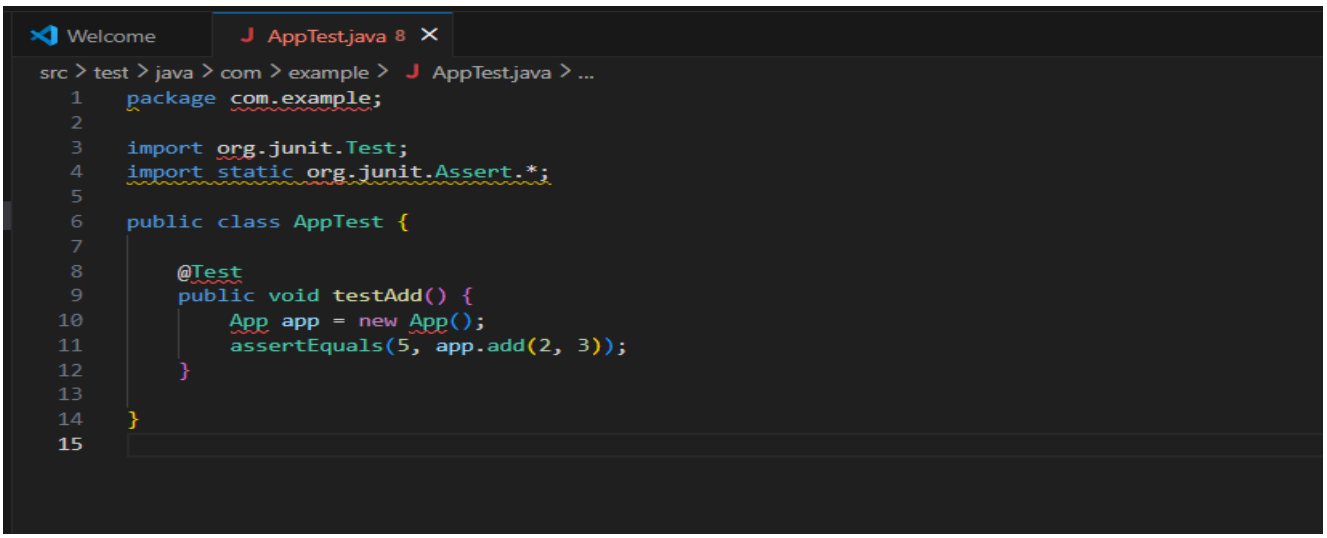


Figure 3: JUnit Test Case Implementation

<u>Step 4:</u> Create pom.xml

File Path:

simple-maven-app/pom.xml

<u>Code:</u>

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
 http://maven.apache.org/xsd/maven-4.0.0.xsd">

 <modelVersion>4.0.0</modelVersion>
 <groupId>com.example</groupId>
 <artifactId>simple-maven-app</artifactId>
 <version>1.0-SNAPSHOT</version>
```

```
<dependencies>
 <dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>4.13.2</version>
  <scope>test</scope>
 </dependency>
</dependencies>


</project>
```

Explanation:

- pom.xml defines project configuration.
- JUnit dependency enables unit testing.
- Maven uses this file to build the project.

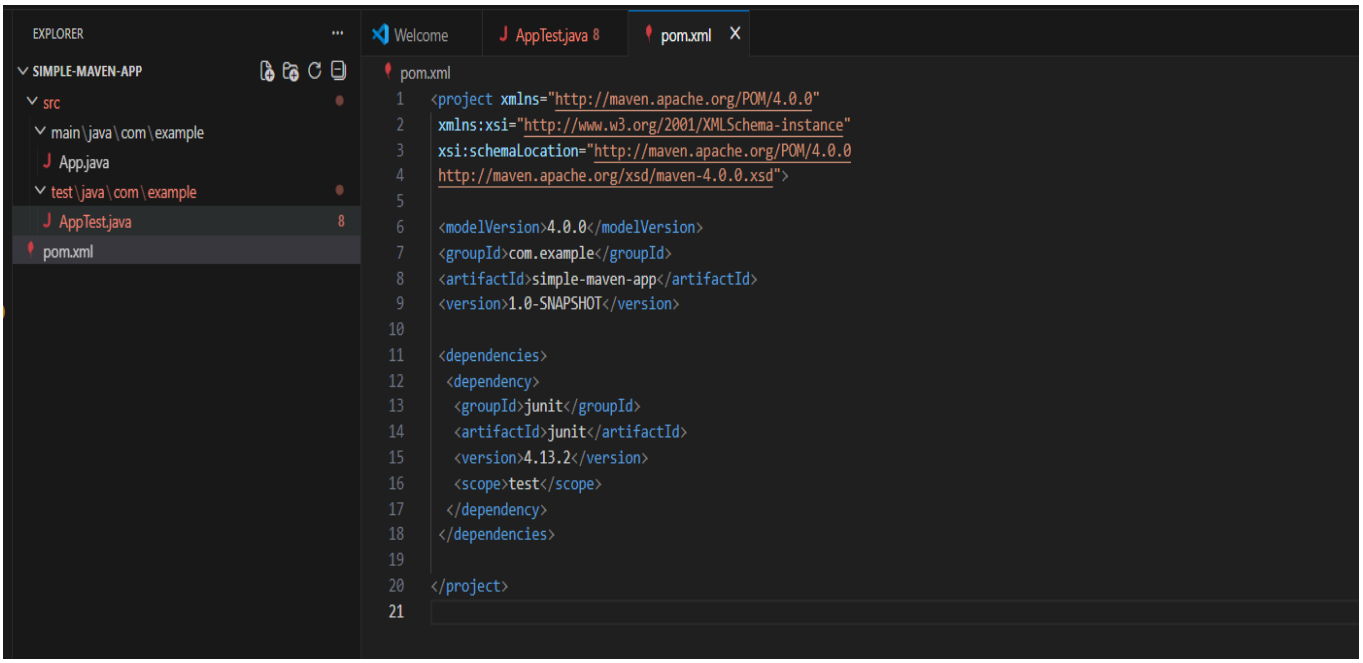


Figure 4: Maven Configuration File (pom.xml)

<u>Step 5:</u> Test Project Locally

Open terminal inside project folder:

mvn clean test

<u>Expected Output:</u>

BUILD SUCCESS

```
PROBLEMS 2   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   AZURE

--- install:3.1.2:install (default-install) @ simple-maven-app ---
Installing C:\Users\sarva\Downloads\simple-maven-app\pom.xml to C:\Users\sarva\.m2\repository\com\example\simple-maven-app\1.0-SNAPSHOT
\simple-maven-app-1.0-SNAPSHOT.pom
Installing C:\Users\sarva\Downloads\simple-maven-app\target\simple-maven-app-1.0-SNAPSHOT.jar to C:\Users\sarva\.m2\repository\com\exam
ple\simple-maven-app\1.0-SNAPSHOT\simple-maven-app-1.0-SNAPSHOT.jar
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  19.450 s
Finished at: 2026-02-07T01:35:57+05:30
------------------------------------------------------------------------

Ln 21, Col 1   Spac
```

Figure 5: Local Maven Build Successful

<u>Step 6:</u> Push Project to GitHub

COMMANDS:

git init

git add README.md

git commit -m "first commit"

git branch -M main

git remote add origin https://github.com/saravanan110220/maven-pipeline.git

git push -u origin main

```
PROBLEMS 2   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   AZURE

C:\Users\sarva\Downloads\simple-maven-app>
C:\Users\sarva\Downloads\simple-maven-app>maven-pipeline
'maven-pipeline' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\sarva\Downloads\simple-maven-app>
C:\Users\sarva\Downloads\simple-maven-app>git remote add origin https://github.com/saravanan110220/maven-pipeline.git

C:\Users\sarva\Downloads\simple-maven-app>
C:\Users\sarva\Downloads\simple-maven-app>git push -u origin main
Enumerating objects: 36, done.
Counting objects: 100% (36/36), done.
Delta compression using up to 12 threads
Compressing objects: 100% (16/16), done.
Writing objects: 100% (36/36), 4.08 KiB | 418.00 KiB/s, done.
Total 36 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/saravanan110220/maven-pipeline.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

C:\Users\sarva\Downloads\simple-maven-app>
C:\Users\sarva\Downloads\simple-maven-app>
```

Figure 6: Maven Project Uploaded to GitHub

<u>Step 7:</u> Install Maven on Windows

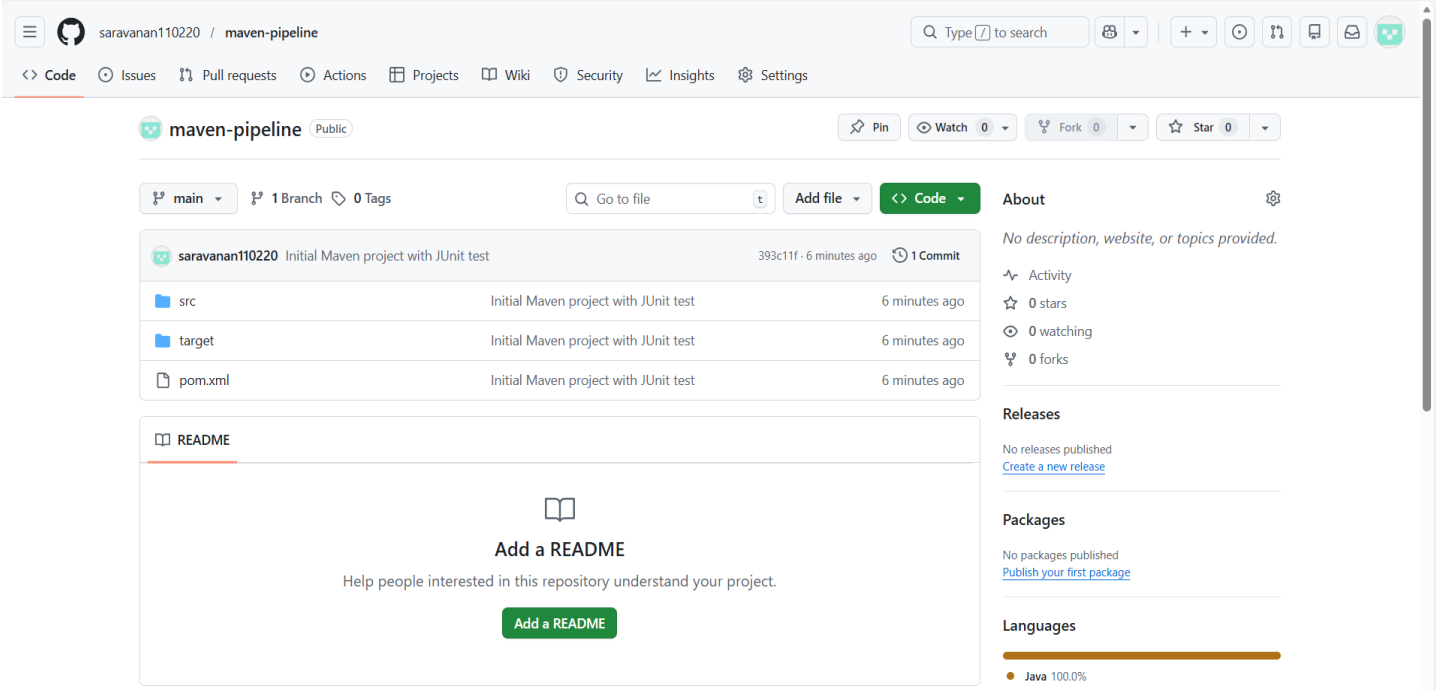
1. Download Apache Maven.
2. **Extract to:**

   C:\Program Files\Apache\apache-maven-3.9.12

3. **Create System Variable:**

   MAVEN_HOME = C:\Program Files\Apache\apache-maven-3.9.12

4. **Add to PATH:**



   %MAVEN_HOME%\bin

5. Verify:

mvn -version

```
Command Prompt

Microsoft Windows [Version 10.0.26200.7705]
(c) Microsoft Corporation. All rights reserved.

C:\Users\sarva>mvn -version
Apache Maven 3.9.12 (848fbb4bf2d427b72bdb2471c22fced7ebd9a7a1)
Maven home: C:\Program Files\apache-maven-3.9.12
Java version: 25, vendor: Eclipse Adoptium, runtime: C:\Program Files\Eclipse Adoptium\jdk-25.0.0.36-hotspot
Default locale: en_IN, platform encoding: UTF-8
OS name: "windows 11", version: "10.0", arch: "amd64", family: "windows"

C:\Users\sarva>D
```

Figure 7: Maven Installed Successfully

<u>Step 8:</u> Configure Maven in Jenkins

Go to:

Manage Jenkins → Tools → Maven Installations

<u>Configure:</u>

- Name: M3
- MAVEN_HOME: C:\Program Files\Apache\apache-maven-3.9.12

- Uncheck Install Automatically

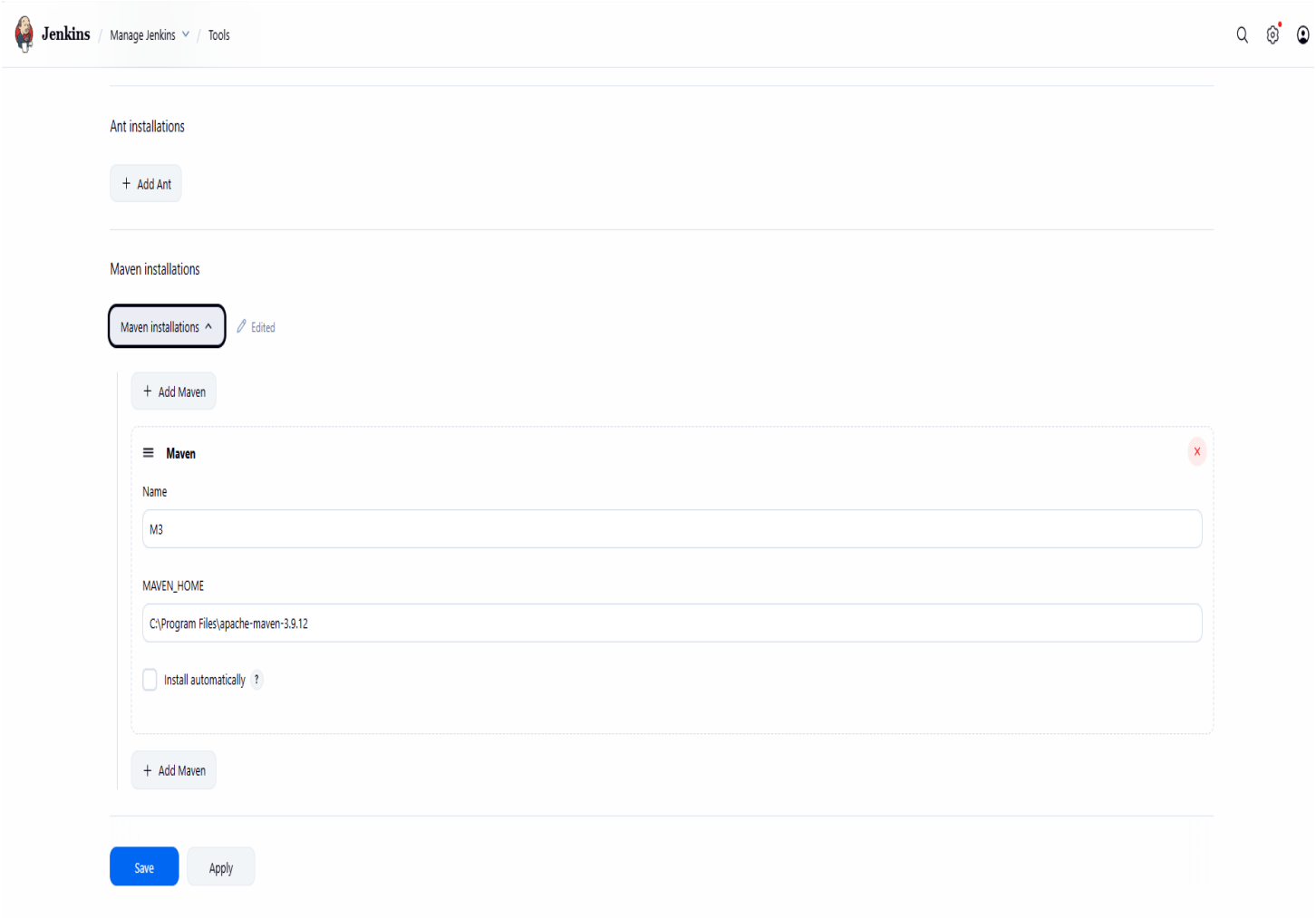


Figure 8: Maven Configured in Jenkins

<u>Step 9:</u> Create Jenkins Pipeline

Create new Pipeline job.

Pipeline Script:

```
pipeline {
  agent any

  tools {
    maven 'M3'
    // 'M3' must match the Maven name you configured in Jenkins Tools
  }

  stages {
    stage('Checkout Source Code') {
      steps {
```

```
            git branch: 'main',
                url: 'https://github.com/saravanan110220/maven-pipeline.git'
        }
    }

    stage('Build & Run Tests') {
        steps {
            echo "Running Maven build and executing JUnit tests..."
            bat 'mvn clean test'
        }
    }
}

post {
    success {
        echo "Pipeline completed successfully!"
    }
    failure {
        echo "Pipeline failed. Check errors above!"
    }
}
}
```

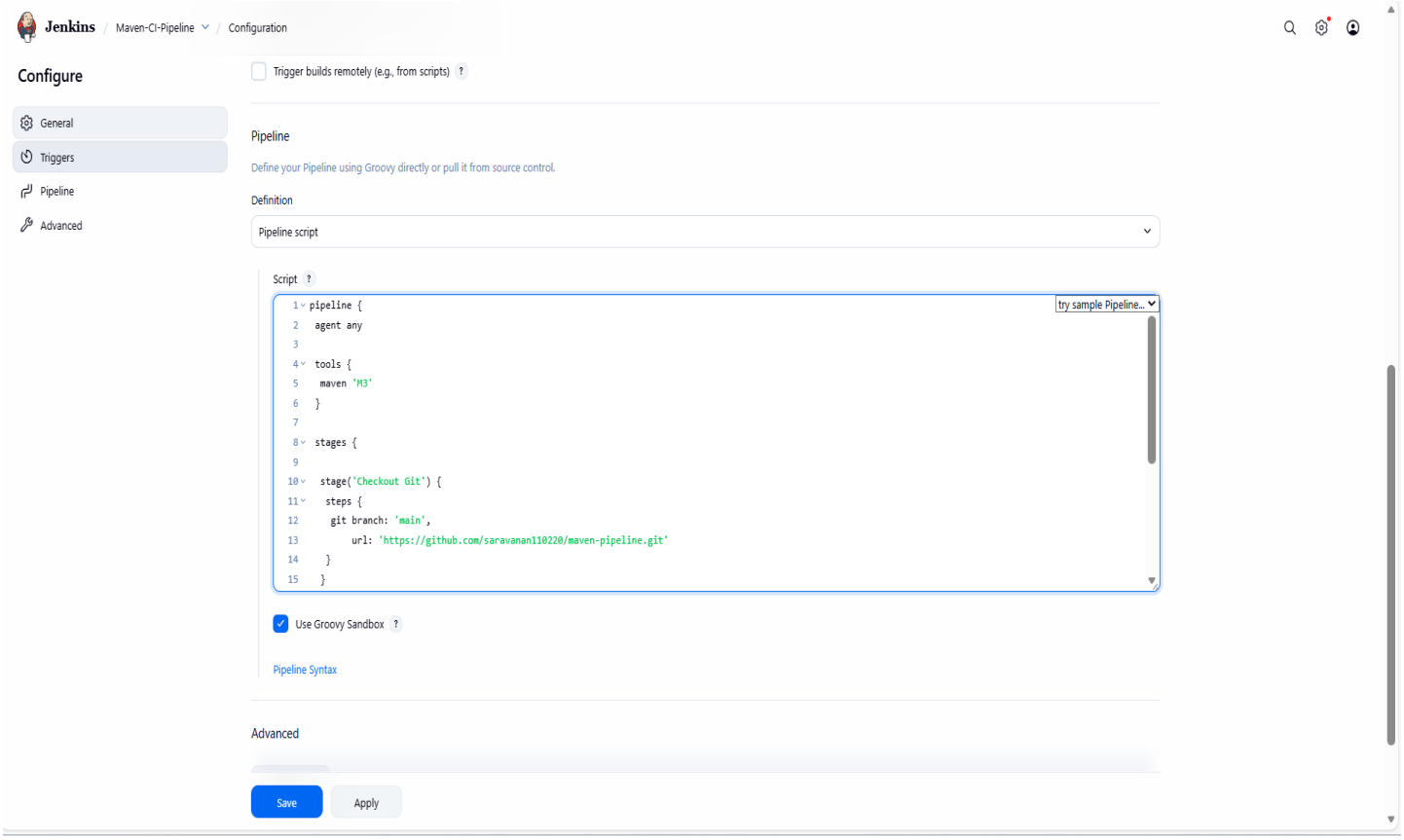
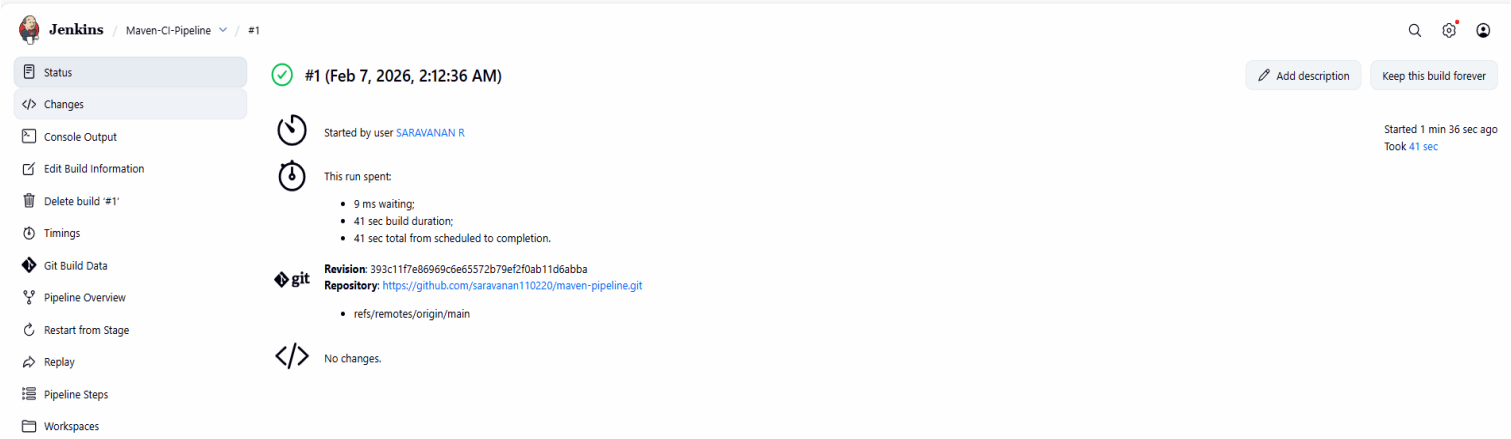


Figure 9: Jenkins Declarative Pipeline Script

<u>Step 10:</u> Run Jenkins Pipeline

1. Click Save.
2. Click Build Now.
3. Open Console Output.



Jenkins / Maven-CI-Pipeline / #1 / Console Output

- Status
- Changes
- Console Output
- Edit Build Information
- Delete build '#1'
- Timings
- Git Build Data
- Pipeline Overview
- Restart from Stage
- Replay
- Pipeline Steps
- Workspaces

Console Output

Download | Copy | View as plain text

```
Started by user SARAVANAN R
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in C:\ProgramData\Jenkins\.jenkins\workspace\Maven-CI-Pipeline
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Tool Install)
[Pipeline] tool
[Pipeline] envVarsForTool
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Checkout Git)
[Pipeline] tool
[Pipeline] envVarsForTool
[Pipeline] withEnv
[Pipeline] {
[Pipeline] git
The recommended git tool is: NONE
No credentials specified
Cloning the remote Git repository
Cloning repository https://github.com/saravanan110220/maven-pipeline.git
 > git.exe init C:\ProgramData\Jenkins\.jenkins\workspace\Maven-CI-Pipeline # timeout=10
Fetching upstream changes from https://github.com/saravanan110220/maven-pipeline.git
 > git.exe --version # timeout=10
 > git --version # 'git version 2.53.0.windows.1'
 > git.exe fetch --tags --force --progress -- https://github.com/saravanan110220/maven-pipeline.git +refs/heads/*:refs/remotes/origin/* # timeout=10
 > git.exe config remote.origin.url https://github.com/saravanan110220/maven-pipeline.git # timeout=10
 > git.exe config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
 > git.exe rev-parse "refs/remotes/origin/main^{commit}" # timeout=10
Checking out Revision 393c11f7e86969c6e65572b79ef2f0ab11d6abba (refs/remotes/origin/main)
 > git.exe config core.sparsecheckout # timeout=10
 > git.exe checkout -f 393c11f7e86969c6e65572b79ef2f0ab11d6abba # timeout=10
```

Jenkins / Maven-CI-Pipeline / #1 / Console Output

```
Progress (3): 12 kB | 18 kB | 7.7/26 kB
Progress (3): 12 kB | 18 kB | 16/26 kB
Progress (4): 12 kB | 18 kB | 16/26 kB | 7.7/18 kB
Progress (4): 12 kB | 18 kB | 16/26 kB | 16/18 kB
Progress (4): 12 kB | 18 kB | 16/26 kB | 18 kB
Progress (4): 12 kB | 18 kB | 26 kB | 18 kB

Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/common-junit3/3.2.5/common-junit3-3.2.5.jar (12 kB at 33 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/surefire-junit4/3.2.5/surefire-junit4-3.2.5.jar (18 kB at 41 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/common-java5/3.2.5/common-java5-3.2.5.jar (18 kB at 37 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/common-junit4/3.2.5/common-junit4-3.2.5.jar (26 kB at 42 kB/s)
[INFO]
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running com.example.AppTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.053 s -- in com.example.AppTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  35.806 s
[INFO] Finished at: 2026-02-07T02:13:16+05:30
[INFO] ------------------------------------------------------------------------
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Figure 10: Successful Jenkins CI Pipeline Execution

Expected:

- Tests run successfully

- BUILD SUCCESS

– Finished: SUCCESS

FINAL RESULT:

- Jenkins checks out the Maven project from GitHub, builds it using Maven, runs JUnit unit tests automatically, and reports build success or failure.
- This completes a full CI pipeline.

EXPERIMENT – 2 :

Jenkin – Docker – Kubernetes (CI-CD Demo):

Step1:

Install Docker Desktop

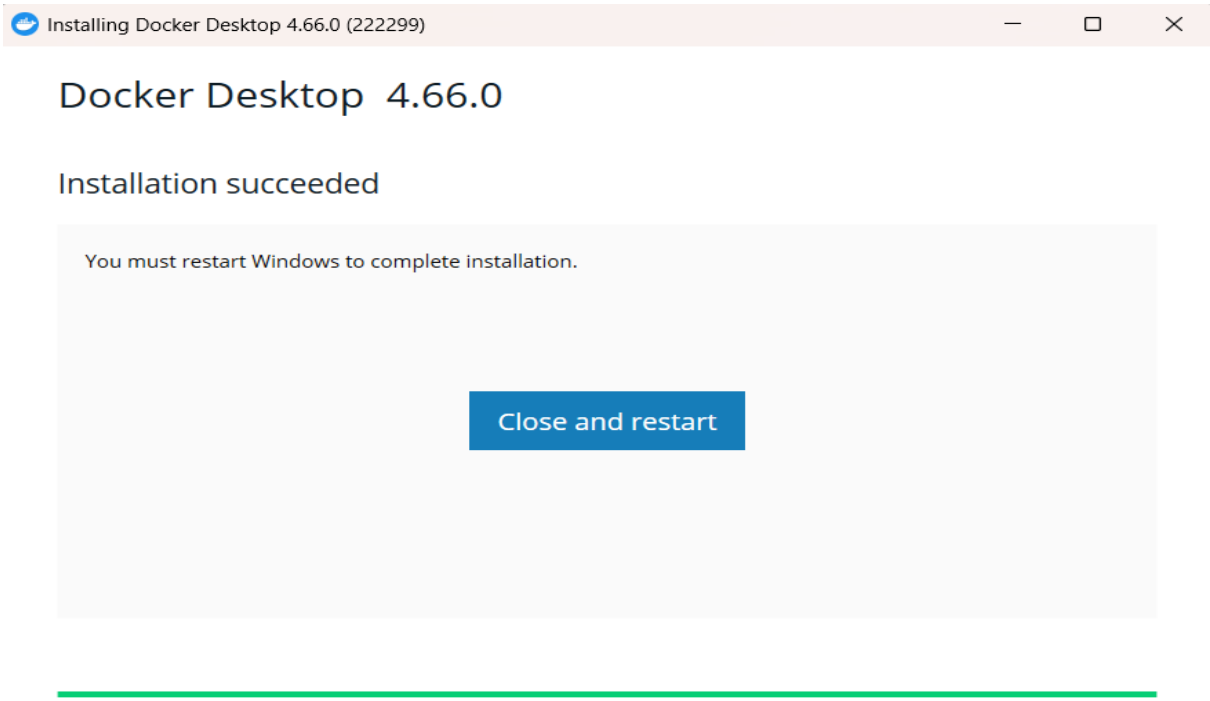


Figure 1: Successful Installation of Docker

Step2 :

Enable Kubernetes

1. **Open the Docker Dashboard**: Launch the Docker Desktop application on your system

2. **Navigate to Settings:** Click the gear icon (Settings or Preferences) in the top-right or sidebar of the Docker Dashboard.

3. **Select the Kubernetes Tab:** In the Settings menu, find and select the **Kubernetes** tab.

4. **Enable Kubernetes:** Check the box labeled **Enable Kubernetes.**

5. **Apply & Restart:** Click the **Apply &** Restart button at the bottom of the settings window. Docker Desktop will then download the necessary Kubernetes components and start a single-node cluster. This may take a few minutes the first time

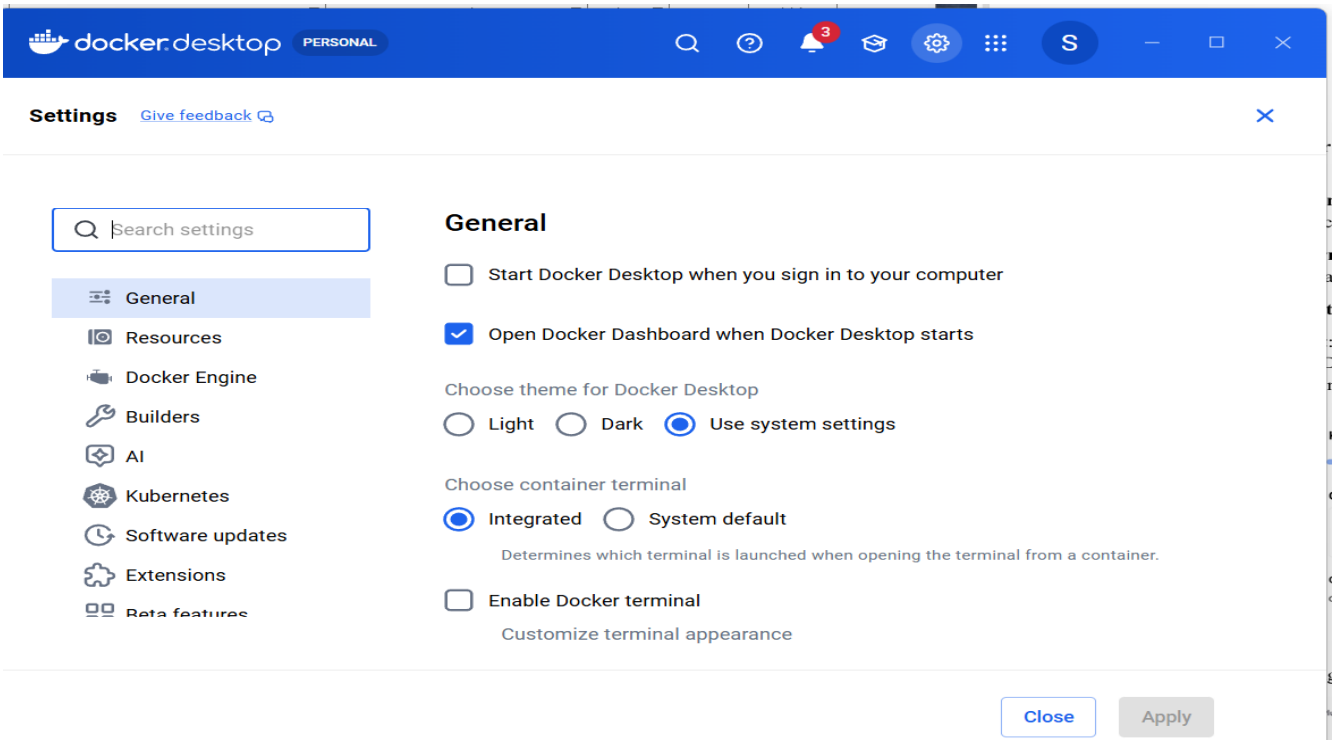


Figure 2 : Navigating to the settings tab

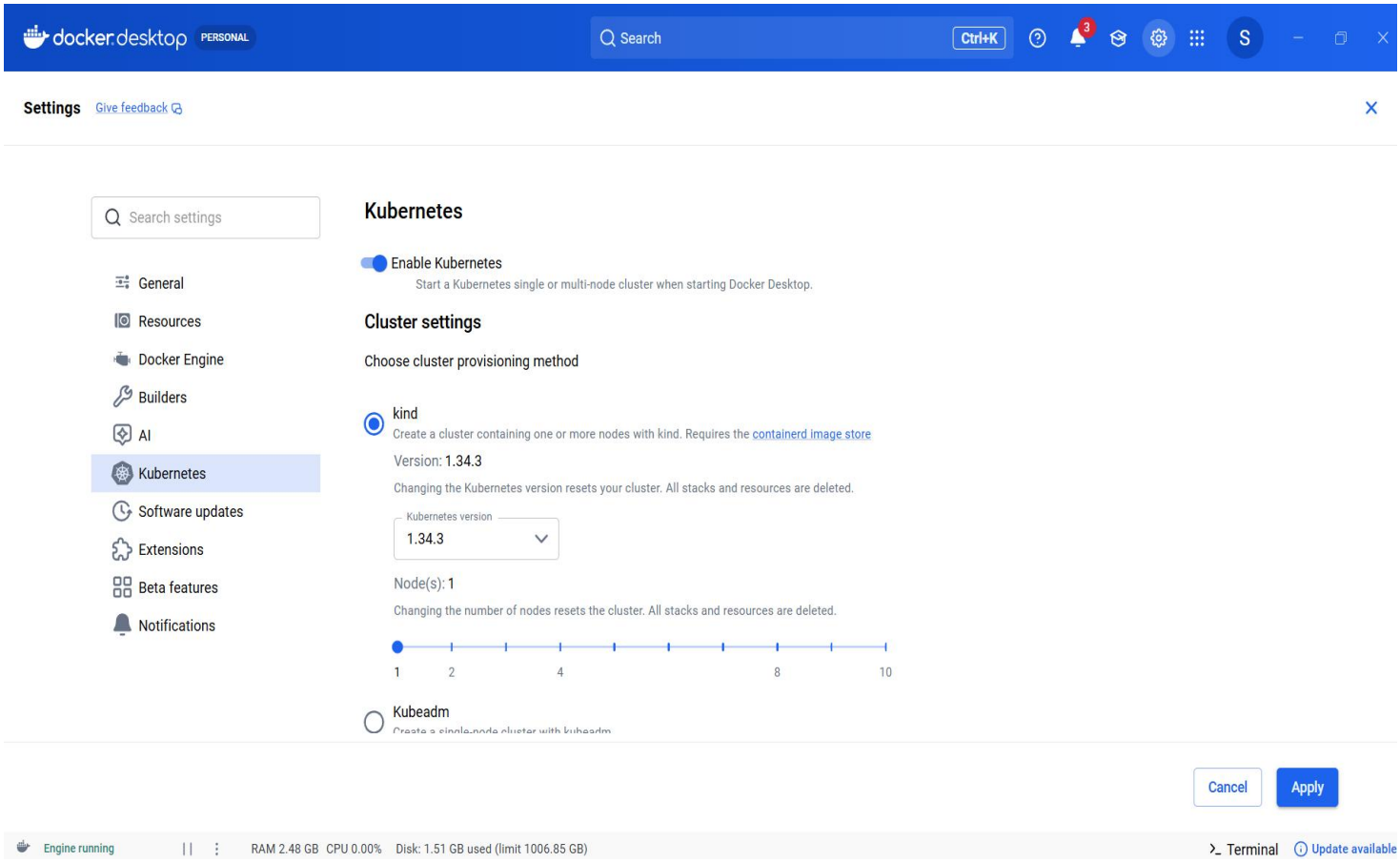


Figure 3: Enabling the Kubernetes

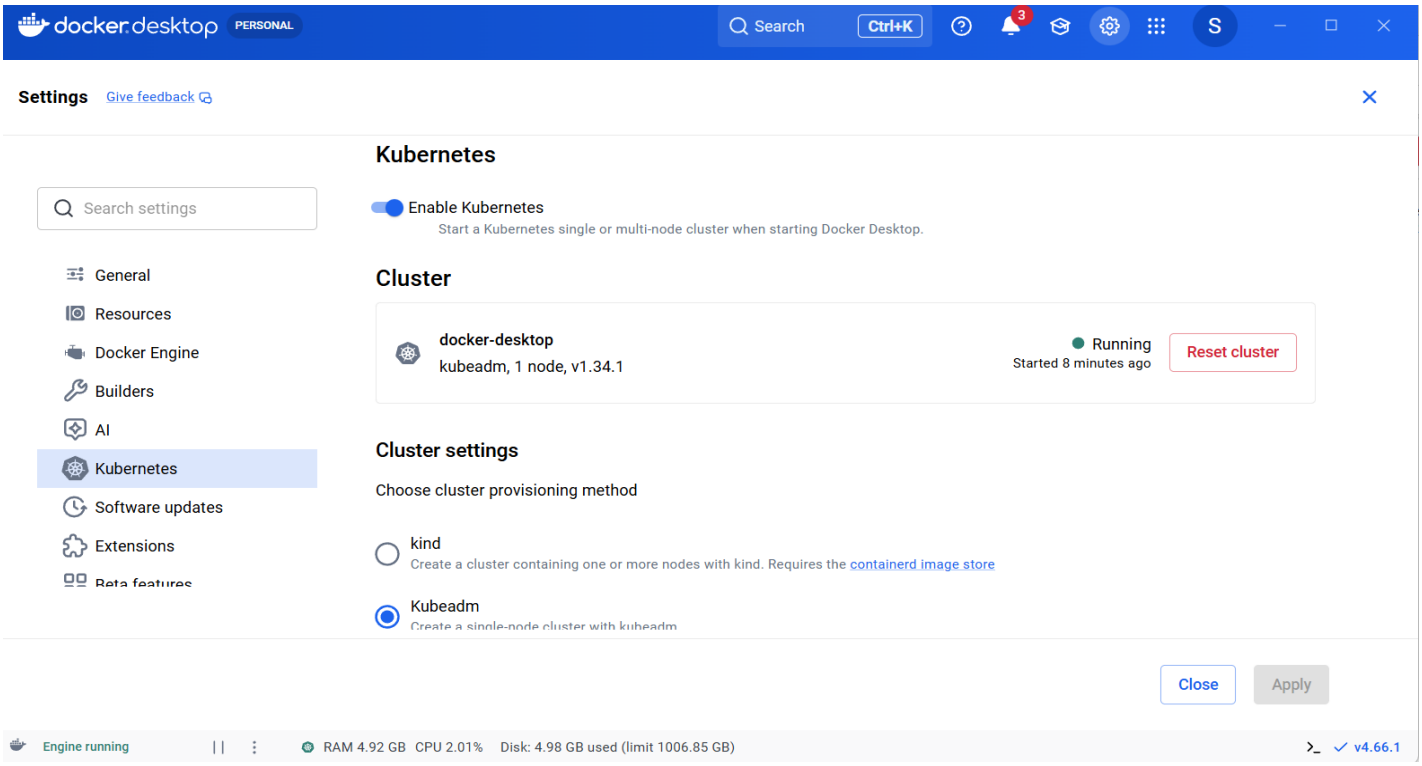


Figure 4: Cluster Started Running

Step 3 :

In Jenkin, enable two plugins (Docker and Docker Pipeline) and Install them

❖ Open Jenkins in browser:

http://localhost:8080

❖ **Install Plugins:**

- Docker
- Docker Pipeline
- Kubernetes CLI

These are required for CI/CD pipeline

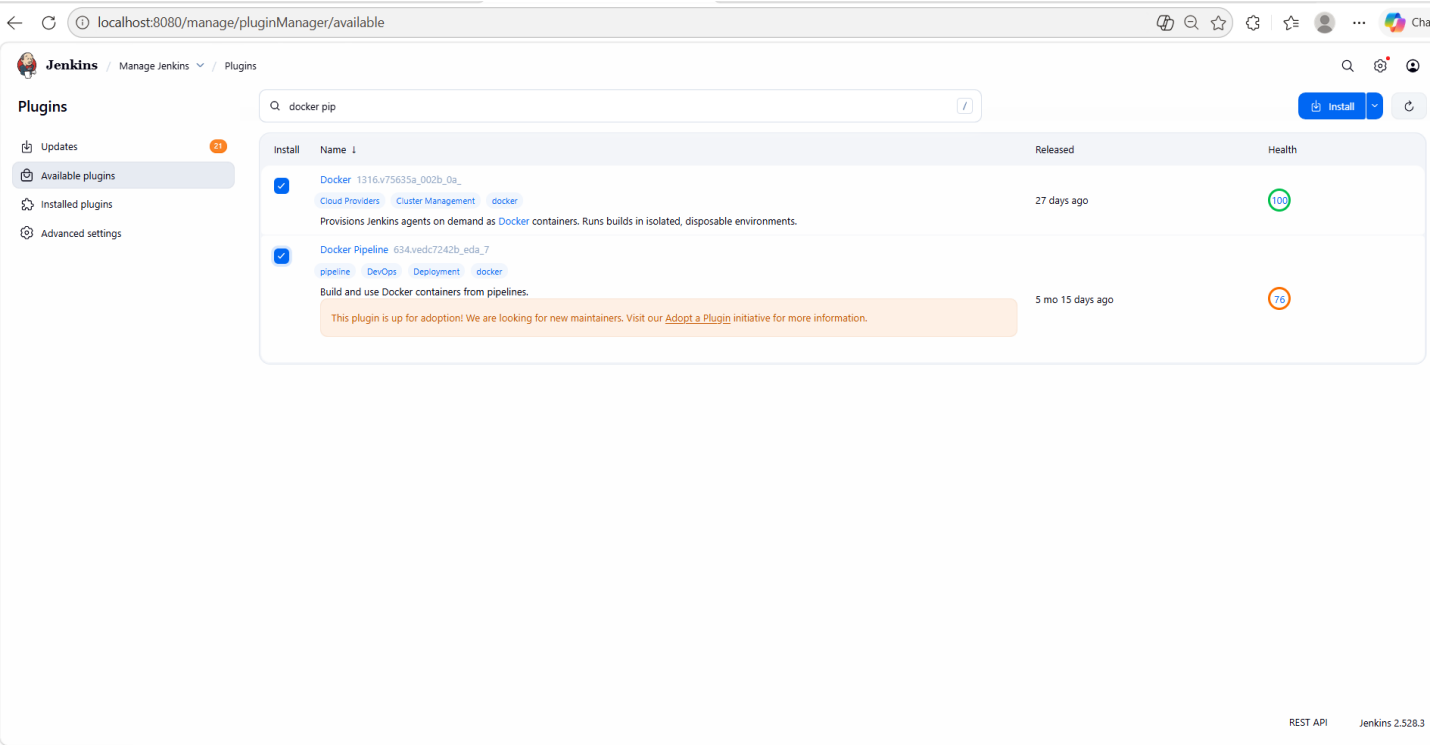


Figure 5: Installing plugins of Docker

❖ **Verify Installation**

1. Go to **Installed Plugins**
2. Check if:
   - Docker ✓
   - Docker Pipeline ✓
   - Kubernetes CLI ✓
3. Ensure status = **Enabled**

✔ Now Jenkins is ready for CI/CD pipeline execution

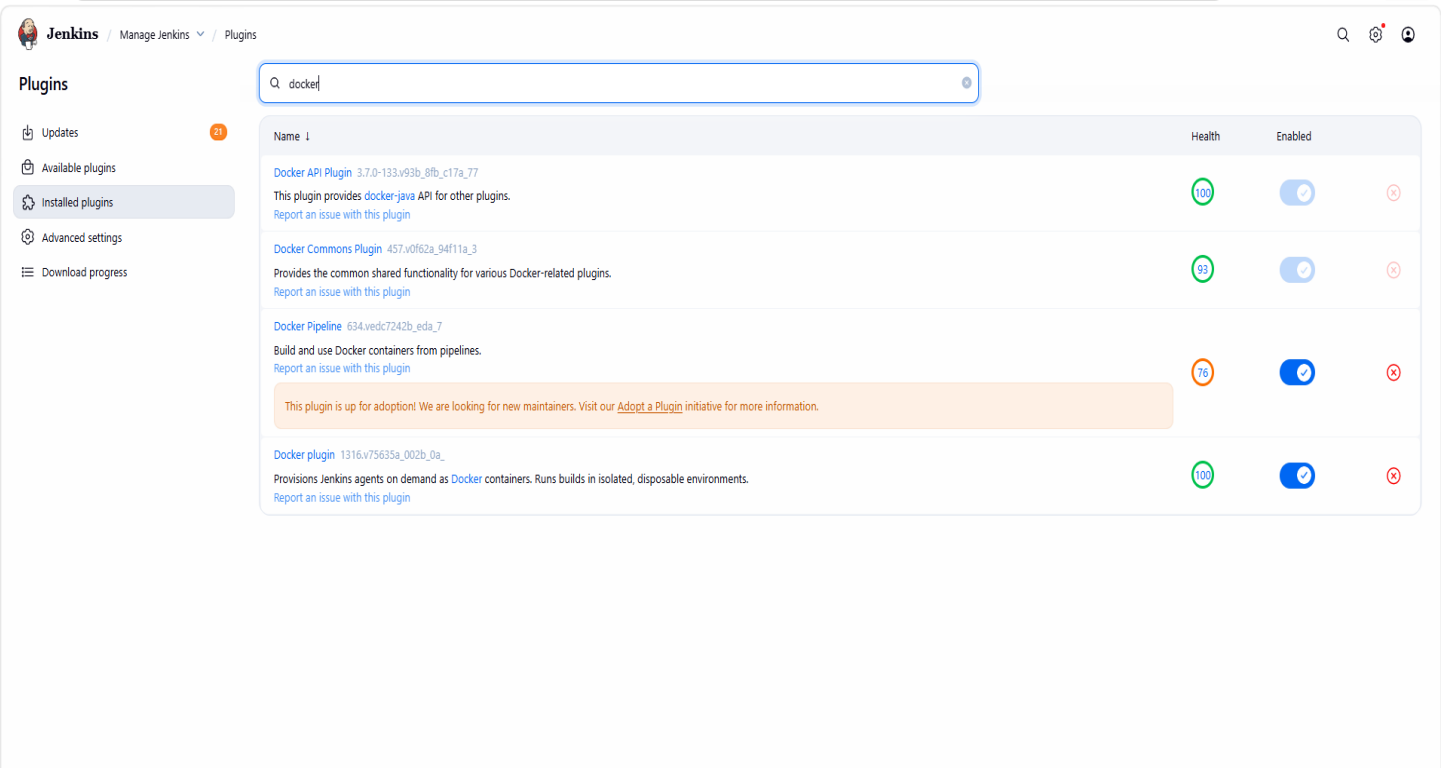


Figure 6: Successfully Installed plugins

<u>Step 4 -</u> Create Simple Java Application

<u>Step 4.1:</u> Create Project Folder

1. Go to your desired location (Desktop or Documents)
2. Create a new folder named:

simple-java-app

✓ This folder will contain all project files

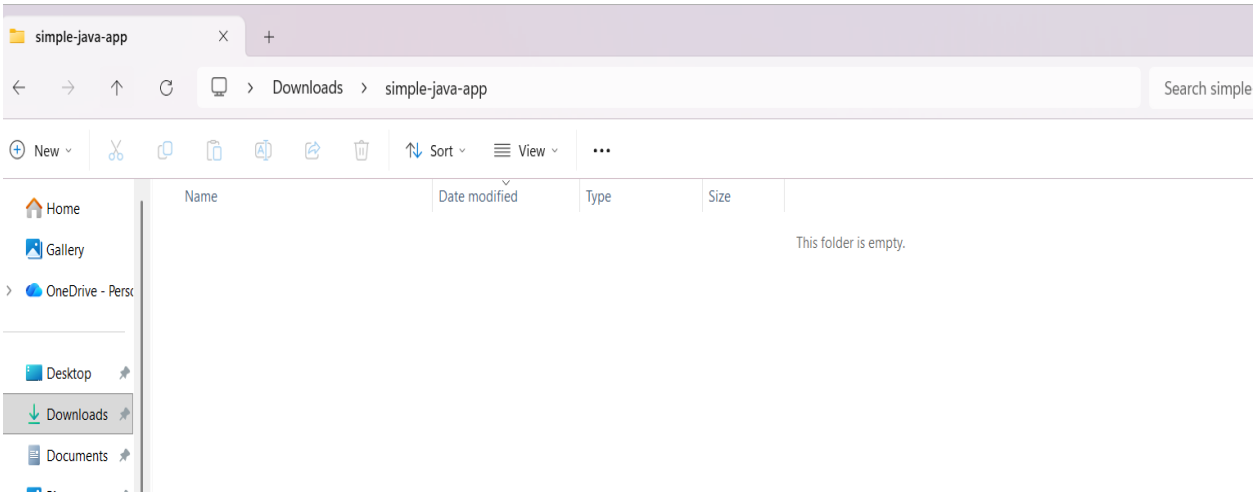


Figure 7: New Folder Created

<u>Step 4.2:</u> Create Java File (App.java)

1. Open the folder
2. Create a new file named:

App.java

3. Add the following code:

```
public class App {
 public static void main(String[] args) throws Exception {
  System.out.println("Java CI/CD Application Started...");

  while (true) {
   Thread.sleep(5000);
   System.out.println("Hello from Java CI/CD Pipeline!");
  }
 }
}
```

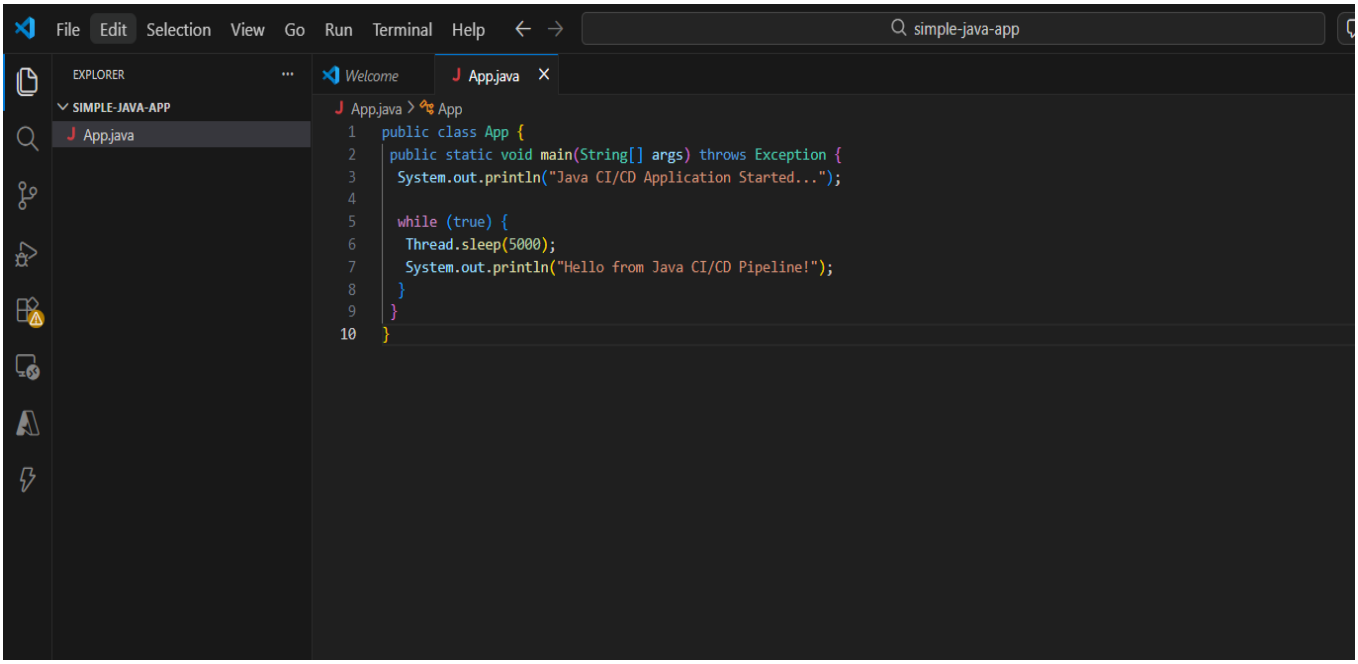


Figure 8 : Java file with Following code

<u>Step 4.3:</u> Create Maven File (pom.xml)

1. In the same folder, create a file:

pom.xml

2. Add the following content:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
 http://maven.apache.org/xsd/maven-4.0.0.xsd">

 <modelVersion>4.0.0</modelVersion>

 <groupId>com.demo</groupId>
 <artifactId>simple-java-app</artifactId>
```

```
<version>1.0</version>

<build>
 <plugins>
  <plugin>
   <groupId>org.apache.maven.plugins</groupId>
   <artifactId>maven-compiler-plugin</artifactId>
   <version>3.10.1</version>
   <configuration>
    <source>17</source>
    <target>17</target>
   </configuration>
  </plugin>
 </plugins>
</build>

</project>
```

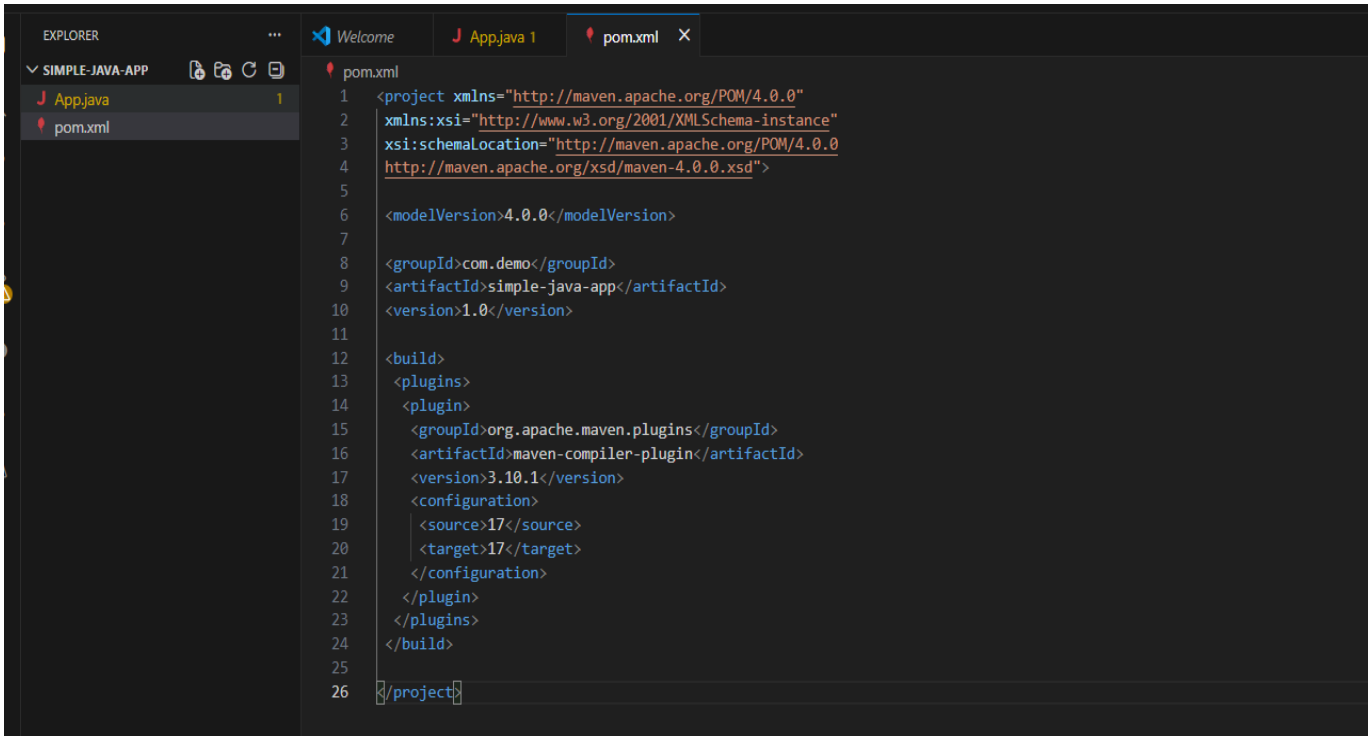


Figure 9 : pom.xml having maven code

<u>Step 5 </u>:Build Locally (Test)

<u>Step 5.1</u>: Open Terminal in Project Folder

1. Navigate to your project folder:

simple-java-app

2. Open:

- Command Prompt (Windows) OR
- PowerShell OR
- Terminal

✓ Make sure you are inside the project directory

```
Command Prompt

Microsoft Windows [Version 10.0.26200.8039]
(c) Microsoft Corporation. All rights reserved.

C:\Users\sarva>cd Downloads

C:\Users\sarva\Downloads>cd simple-java-app

C:\Users\sarva\Downloads\simple-java-app>
```

Figure 10: Project Folder Opened in Terminal

<u>Step 5.2</u>: Run Maven Build Command

Execute the following command:

mvn clean package

```
C:\Users\sarva\Downloads\simple-java-app>mvn clean package
[INFO] Scanning for projects...
[INFO]
[INFO] ----------------------< com.demo:simple-java-app >----------------------
[INFO] Building simple-java-app 1.0
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-clean-plugin/3.2.0/maven-clean-plugin-3.2.0.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-clean-plugin/3.2.0/maven-clean-plugin-3.2.0.pom (5.3 kB at 8.8
kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-plugins/35/maven-plugins-35.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-plugins/35/maven-plugins-35.pom (9.9 kB at 98 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-parent/35/maven-parent-35.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-parent/35/maven-parent-35.pom (45 kB at 520 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/apache/25/apache-25.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/apache/25/apache-25.pom (21 kB at 270 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-compiler-plugin/3.10.1/maven-compiler-plugin-3.10.1.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-compiler-plugin/3.10.1/maven-compiler-plugin-3.10.1.pom (13 kB
at 156 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-plugins/34/maven-plugins-34.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-plugins/34/maven-plugins-34.pom (11 kB at 155 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-compiler-plugin/3.10.1/maven-compiler-plugin-3.10.1.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-compiler-plugin/3.10.1/maven-compiler-plugin-3.10.1.jar (62 kB
at 595 kB/s)
[INFO]
[INFO] --- clean:3.2.0:clean (default-clean) @ simple-java-app ---
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-shared-utils/3.3.4/maven-shared-utils-3.3.4.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-shared-utils/3.3.4/maven-shared-utils-3.3.4.pom (5.8 kB at 91 kB
/s)
Downloading from central: https://repo.maven.apache.org/maven2/commons-io/commons-io/2.6/commons-io-2.6.pom
Downloaded from central: https://repo.maven.apache.org/maven2/commons-io/commons-io/2.6/commons-io-2.6.pom (14 kB at 193 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-parent/42/commons-parent-42.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-parent/42/commons-parent-42.pom (68 kB at 849 kB/s)
```

```
at 6.2 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-utils/3.4.1/plexus-utils-3.4.1.jar (264 kB at 351
Downloaded from central: https://repo.maven.apache.org/maven2/org/ow2/asm/asm/9.2/asm-9.2.jar (122 kB at 161 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-compiler-javac/2.11.1/plexus-compiler-javac-2.11.
 kB/s)
[INFO] No sources to compile
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ simple-java-app ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory C:\Users\sarva\Downloads\simple-java-app\src\test\resources
[INFO]
[INFO] --- compiler:3.10.1:testCompile (default-testCompile) @ simple-java-app ---
[INFO] No sources to compile
[INFO]
[INFO] --- surefire:3.2.5:test (default-test) @ simple-java-app ---
[INFO] No tests to run.
[INFO]
[INFO] --- jar:3.4.1:jar (default-jar) @ simple-java-app ---
[WARNING] JAR will be empty - no content was marked for inclusion!
[INFO] Building jar: C:\Users\sarva\Downloads\simple-java-app\target\simple-java-app-1.0.jar
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  8.832 s
[INFO] Finished at: 2026-03-27T23:59:55+05:30
[INFO] ------------------------------------------------------------------------

C:\Users\sarva\Downloads\simple-java-app>
```

Figure 11 : Maven Build Process Output

<u>Step 5.3:</u> Verify Build Success

After execution, you should see:

BUILD SUCCESS

✓ This indicates:

- No compilation errors
- Project built successfully

<u>Step 5.4:</u> Check Generated .jar File

1. Open your project folder
2. Navigate to:

target/

3. You will find:

simple-java-app-1.0.jar

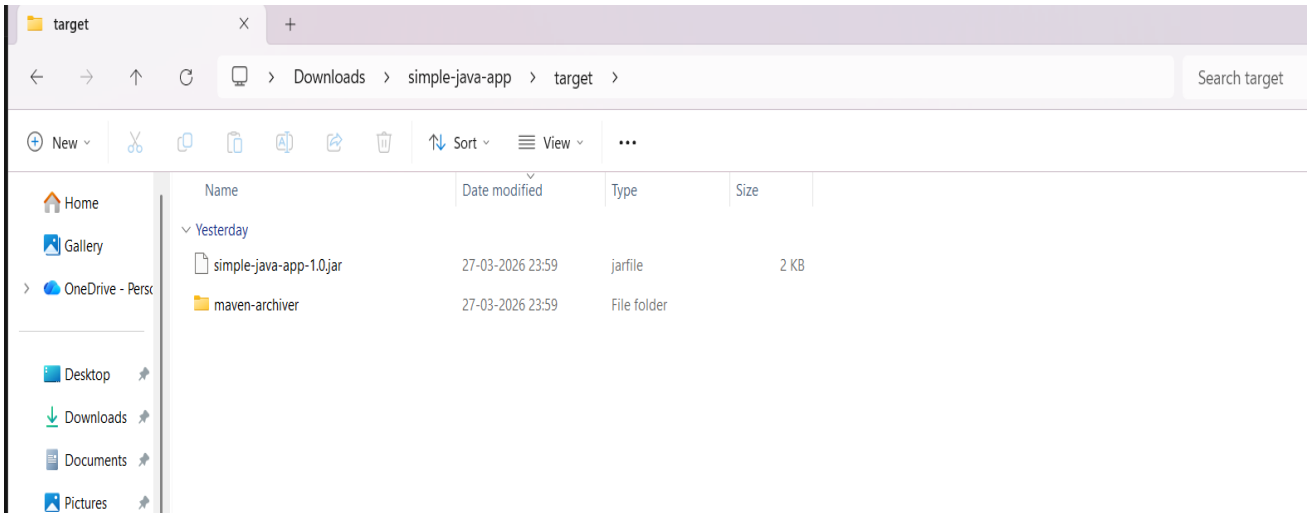


Figure 12 : Generated .jar File

<u>Step 6 -</u> Create Dockerfile

<u>Step 6.1</u>: Ensure Prerequisites

Before creating Dockerfile, confirm:

✓ simple-java-app-1.0.jar is available inside:

target/

✓ Docker Desktop is running

<u>Step 6.2:</u> Create Dockerfile

1. Open your project folder:

simple-java-app

2. Create a new file named:

Dockerfile

<u>Step 6.3:</u> Add Dockerfile Content

Paste the following code:

FROM eclipse-temurin:17-jdk

WORKDIR /app

COPY target/simple-java-app-1.0.jar app.jar

CMD ["java", "-jar", "app.jar"]

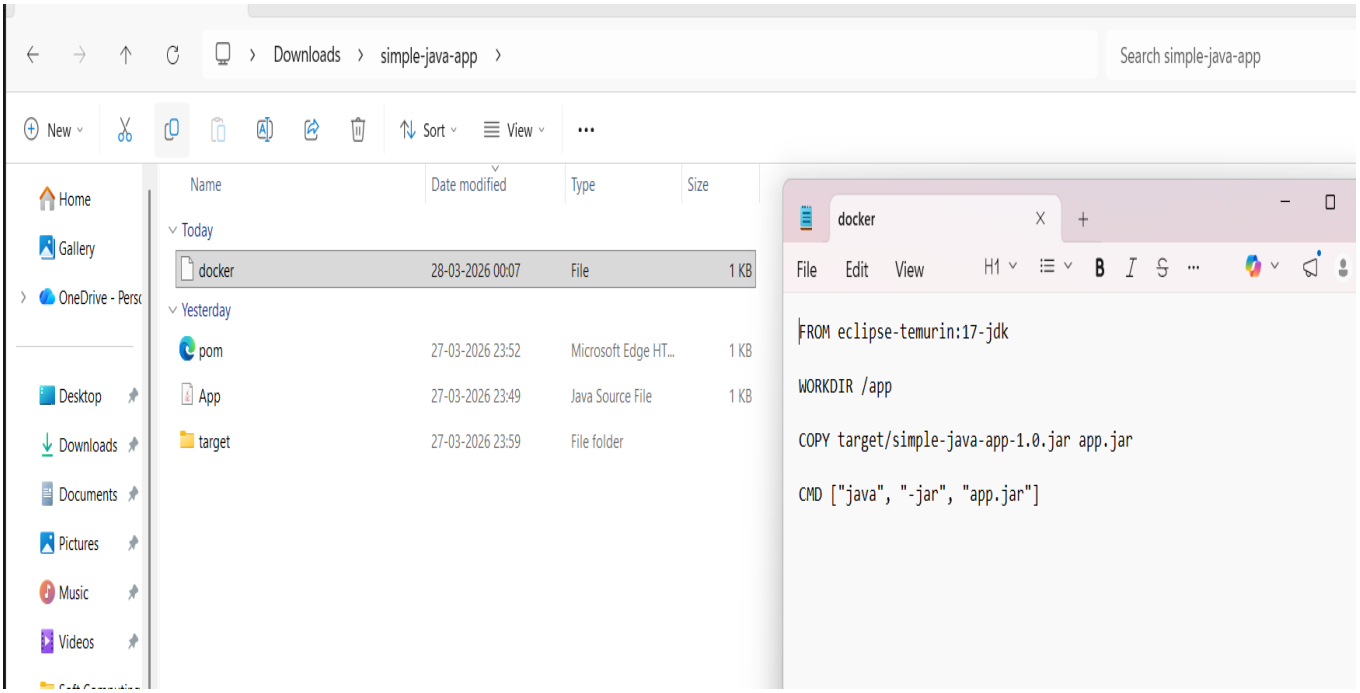


Figure 13 : Created docker file

<u>Step 7 -</u> Push Code to GitHub

<u>Step 7.1:</u> Create Repository in GitHub

1. Go to **GitHub website**
2. Click **New Repository**
3. Enter:
    - Repository Name → simple-java-app
4. Select:
    - Public (recommended for demo)
5. Click **Create Repository**

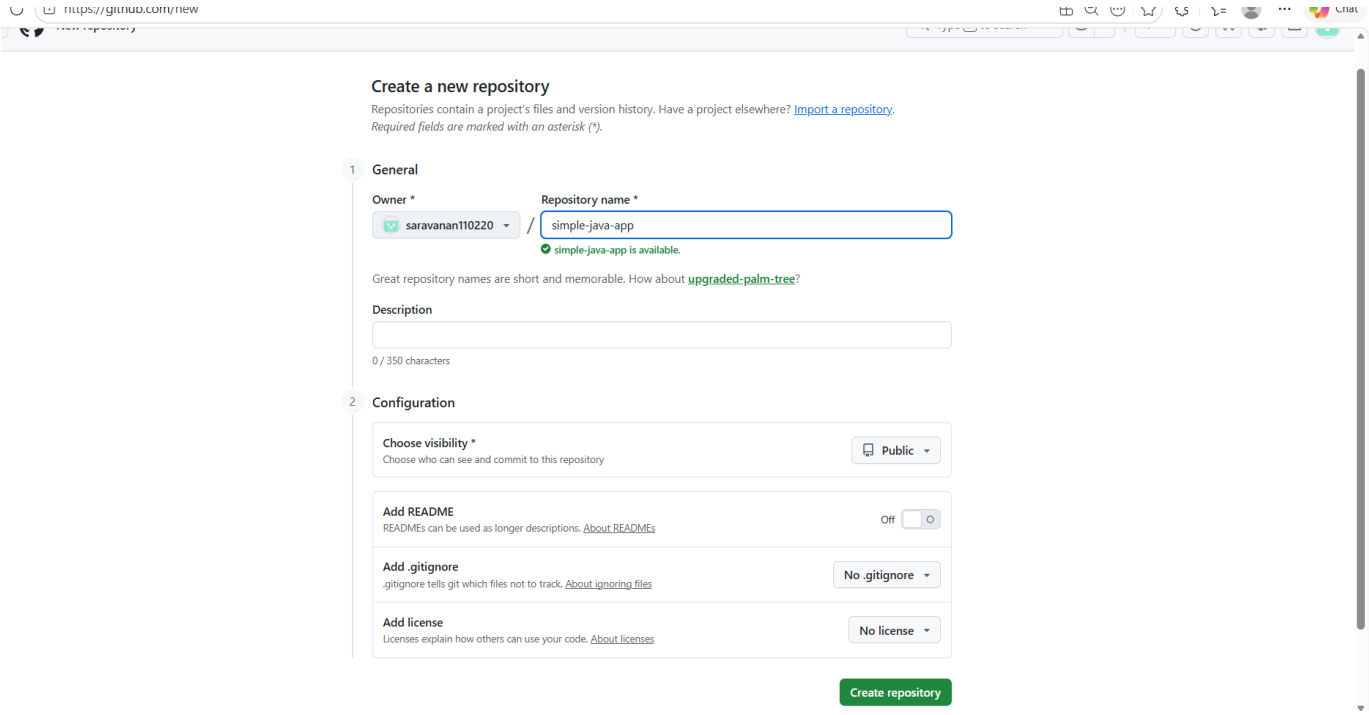


Figure 14 : Github Repository Created

<u>Step 7.2:</u> Initialize Git in Local Project

Open terminal inside project folder:

```
git init
```

✓ This creates a hidden .git folder
✓ Converts project into Git repository

<u>Step 7.3:</u> Add Files to Git

```
git add .
```

✓ Adds all project files:

- App.java
- pom.xml
- Dockerfile
- deployment.yaml

Step 7.4: **Commit Changes**

git commit -m "Initial Java CI/CD commit"

✓ Saves snapshot of project
✓ Message describes changes

Step 7.5: **Set Main Branch**

git branch -M main

✓ Renames branch to main

Step 7.6: **Connect to GitHub Repository**

git remote add origin https://github.com/your-username/simple-java-app.git

✓ Links local project to GitHub

Step 7.7: **Push Code to GitHub**

git push -u origin main

✓ Uploads project to GitHub
✓ Makes it accessible for Jenkins

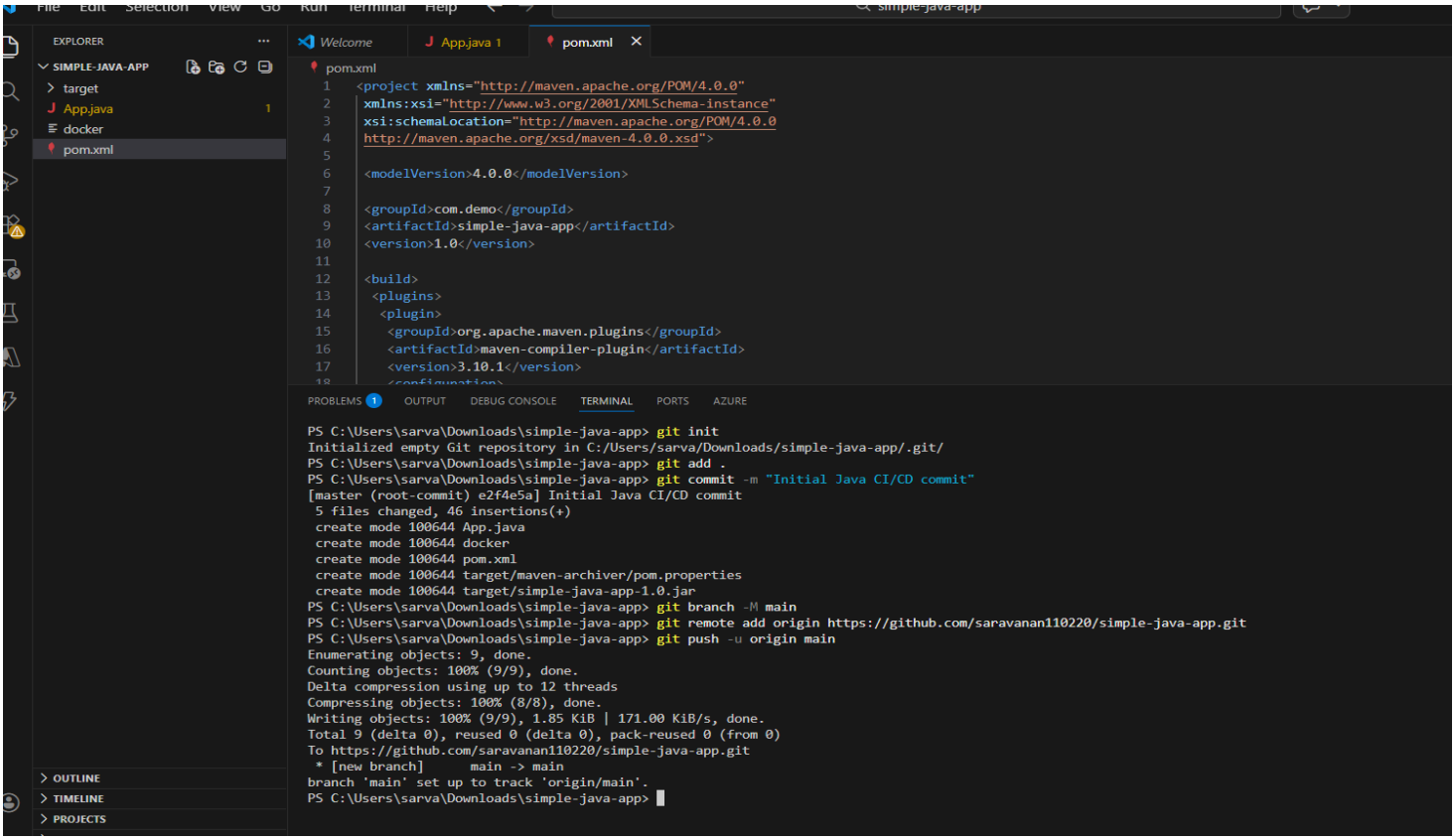


Figure 15 : Folder Pushed to Github

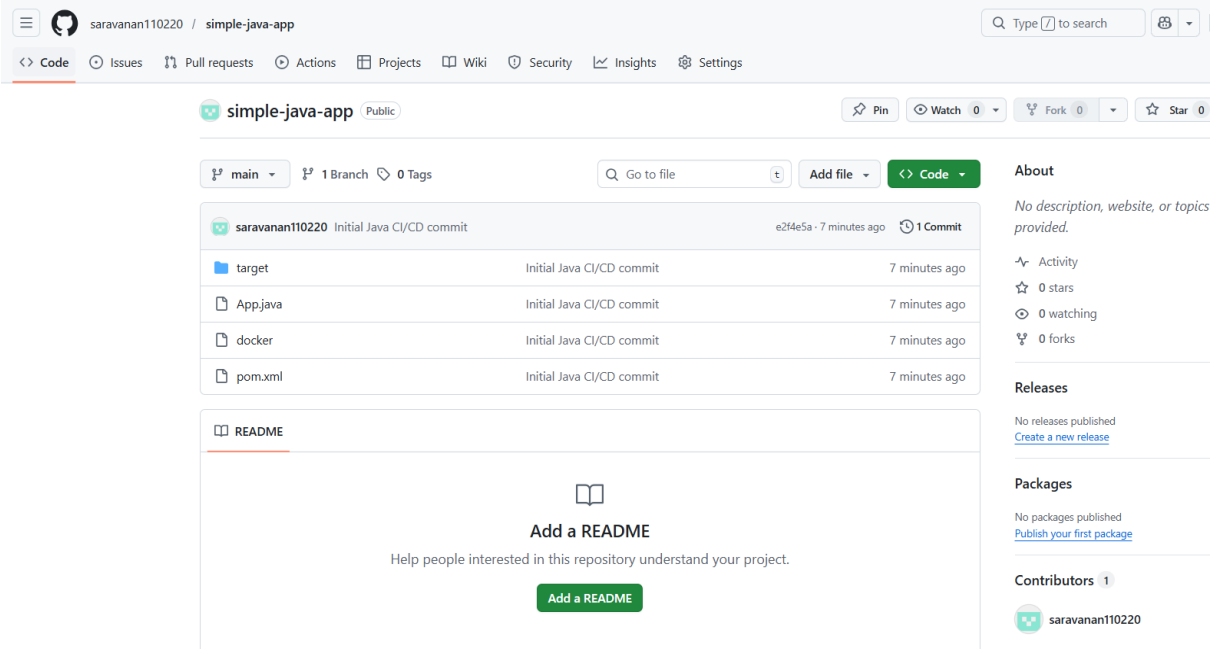


Figure 15 : Files in Github Repository

<u>Step 8</u> - Create Kubernetes Deployment File

<u>Step 8.1:</u> Create deployment.yaml File

1. Open your project folder:

simple-java-app

2. Create a new file:

deployment.yaml

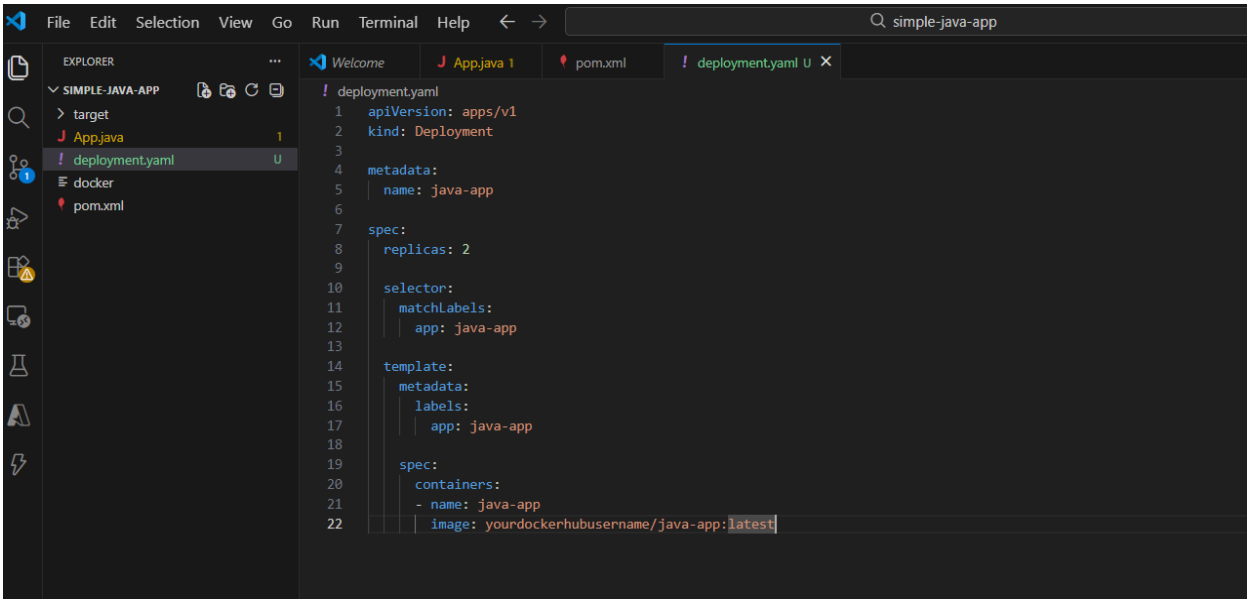


Figure 16 : Created YAML File

<u>Step 8.2:</u> Add YAML Configuration

Paste the following code:

```
apiVersion: apps/v1
kind: Deployment

metadata:
  name: java-app

spec:
  replicas: 2

  selector:
    matchLabels:
      app: java-app

  template:
    metadata:
      labels:
        app: java-app

    spec:
      containers:
      - name: java-app
        image: /java-app:latest
```

<u>Step 8.3:</u> Add Service Configuration

Below the above code, add:

```
---
apiVersion: v1
kind: Service

metadata:
  name: java-service

spec:
  type: NodePort

  selector:
    app: java-app

  ports:
  - port: 80
    targetPort: 8080
    nodePort: 30008
```

Step 8.4 : After creating it, Push it to GitHub:

git add deployment.yaml

git commit -m "Added Kubernetes deployment file"

git push

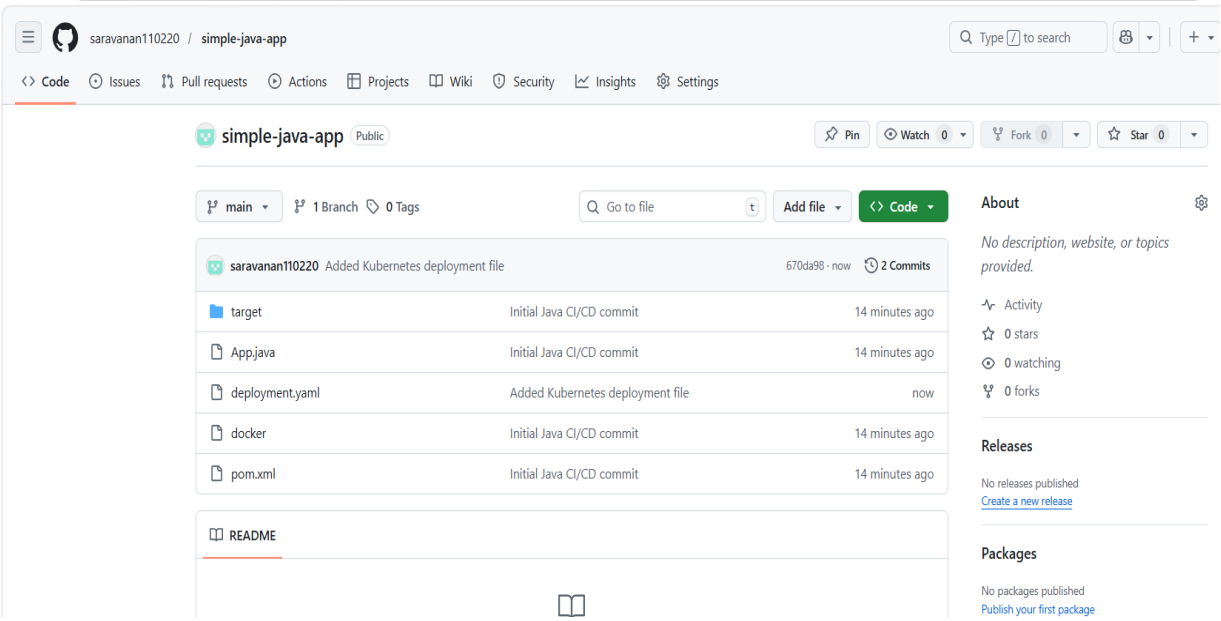


Figure 17: Pushed the deployment.yaml

Step 9 - Configure Jenkins

Step 9.1: Start Jenkins Server

If Jenkins is installed using .war file:

java -jar jenkins.war

✓ Jenkins starts on:

http://localhost:8080

Step 9.2: Install Suggested Plugins

1. Click:
   👉 **Install Suggested Plugins**

✓ Jenkins installs:

- Git
- Pipeline
- Basic dependencies

<u>Step 9.3:</u> Create Admin User

1. Enter:
    - Username
    - Password
    - Email
2. Click:
    👉 **Save and Continue**

    ✓ Jenkins setup completes

<u>Step 9.4 :</u> Run Jenkins

Install Kubernetes CLI plugin

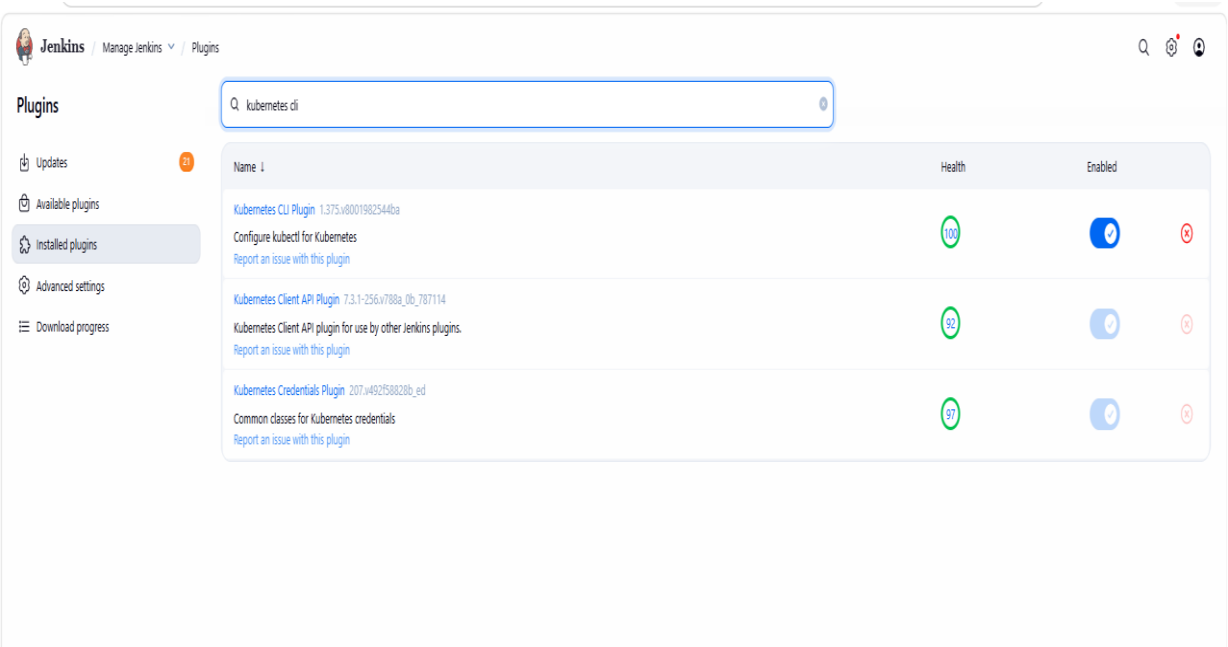


Figure 18 : Installed Kubernetes Plugin

<u>Step 10 -</u> Add DockerHub Credentials

<u>Step 10.1:</u> Open Jenkins Dashboard

1. Open browser:

http://localhost:8080

2. Login to Jenkins

<u>Step 10.2:</u> Navigate to Credentials Section

1. Click:
    👉 **Manage Jenkins**
2. Click:
    👉 **Manage Credentials**

3. Select:
   👉 **(System)**

4. Click:
   👉 **Global credentials (unrestricted)**

Fill like this:

Field	Value to Enter
Kind	Username with password
Scope	Global
Username	*your DockerHub username*
Password	*your DockerHub password*
ID	dockerhub-creds
Description	DockerHub Login

Click Create

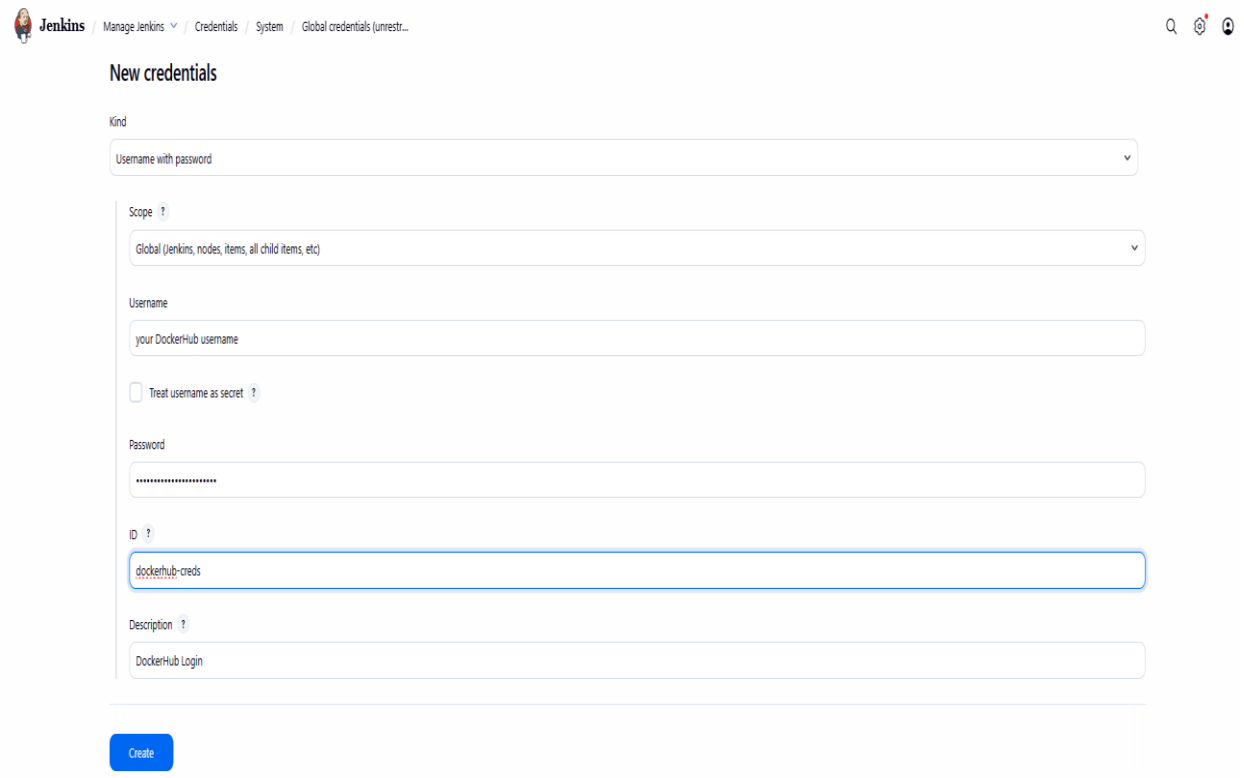


Figure 19: Added credentials

Step 11 - Create Jenkins Pipeline Job

Step 11.1: Create New Pipeline Job

1. Open Jenkins Dashboard
2. Click:
   👉 **New Item**
3. Enter name:

Java-CICD

4. Select:
   👉 **Pipeline**
5. Click:
   👉 **OK**

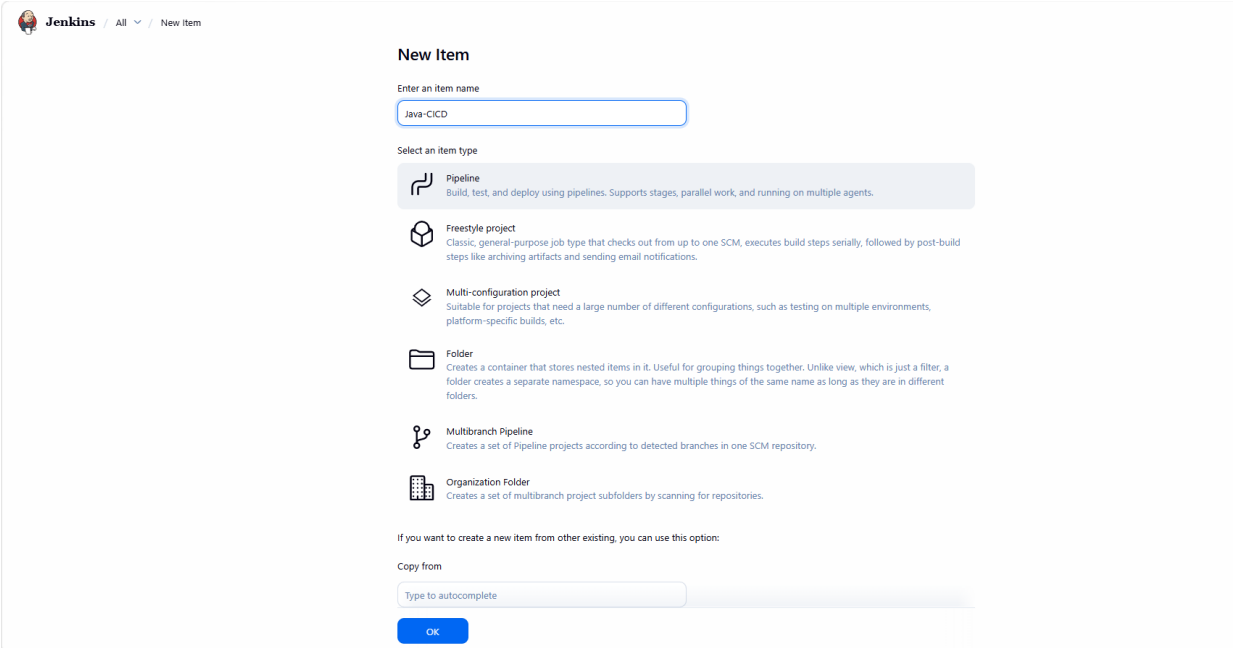


Figure 20: Pipeline Created Successfully

Step 11.2: Configure Pipeline

1. Scroll down to:
   👉 **Pipeline Section**
2. Under **Definition**, select:
   👉 **Pipeline script**

<u>Step 11.3:</u> Add Pipeline Script

Paste the following code:

```
pipeline {
 agent any

 environment {
  DOCKER_IMAGE = "saravanan110220/java-app:latest"
 }

 stages {

  stage('Clone Repository') {
   steps {
    git branch: 'main',
    url: 'https://github.com/ saravanan110220/simple-java-app.git'
   }
  }

  stage('Build with Maven') {
   steps {
    bat 'mvn clean package'
   }
  }

  stage('Build Docker Image') {
   steps {
    bat "docker build -t %DOCKER_IMAGE% ."
   }
  }

  stage('Push to DockerHub') {
   steps {
    withCredentials([usernamePassword(
     credentialsId: 'dockerhub-creds',
     usernameVariable: 'DOCKER_USER',
     passwordVariable: 'DOCKER_PASS'
    )]) {
     bat """
     docker login -u %DOCKER_USER% -p %DOCKER_PASS%
     docker push %DOCKER_IMAGE%
     """
    }
   }
  }

  stage('Deploy to Kubernetes') {
   steps {
    bat 'kubectl apply -f deployment.yaml'
   }
  }

 }

 post {
  success {
   echo 'CI/CD Pipeline executed successfully!'
```

```
    }
    failure {
     echo 'Pipeline failed. Please check logs.'
    }
   }
}
```

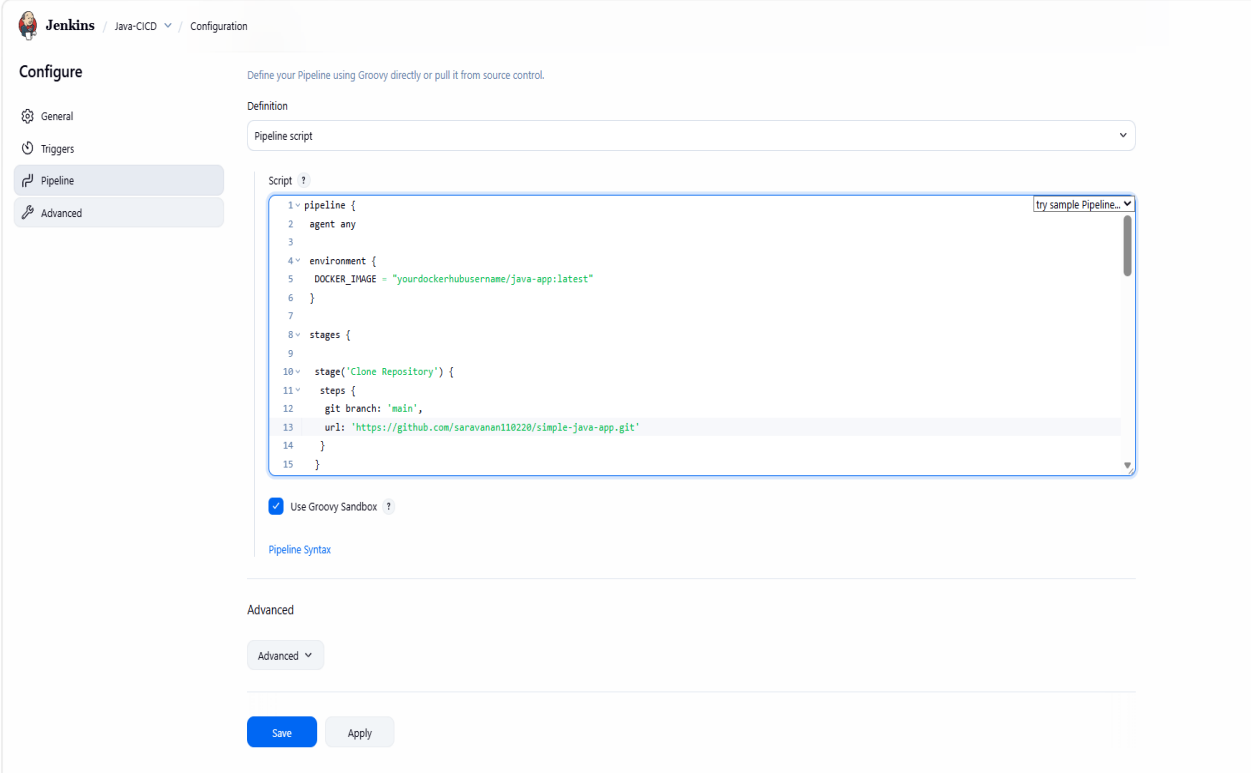


Figure 21: Pipeline Script Created

<u>Step 11.4</u>: Save and Build

1. Click:
   👉 **Save**

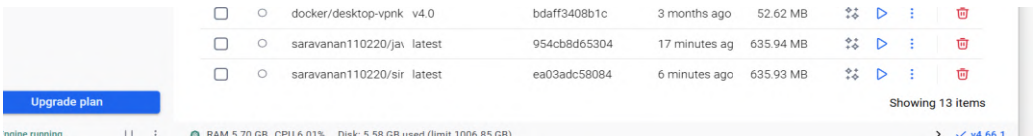
2. Click:
   👉 **Build Now**

```
service/java-service created
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
CI/CD Pipeline executed successfully!
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Figure 22: pipeline successfully executed

Step 12 - Verify Docker and Kubernetes Deployment

Docker Image



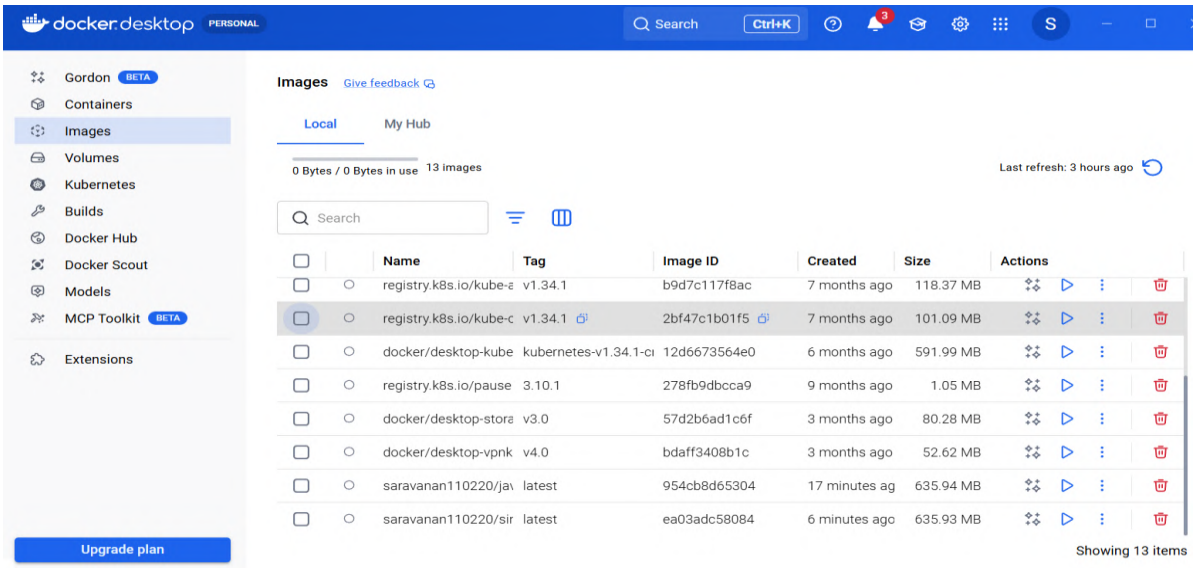


Figure 23: Image created successfully

Kubernetes

Run this command to verify in command prompt,

kubectl get svc

```
C:\Users\sarva\Downloads\simple-java-app>kubectl get svc
NAME           TYPE       CLUSTER-IP      EXTERNAL-IP   PORT(S)
java-service   NodePort   10.111.227.130  <none>        80:30008/TCP
kubernetes     ClusterIP  10.96.0.1       <none>        443/TCP
```